Đề 06 · Nhập môn Công nghệ nền tảng (RIPT1304) · Tài liệu học nội bộ, không nộp

Hiểu và làm chủ phần việc của Chu Hồng Đức

Sinh viên 4 · SRE: sao lưu, khôi phục, diễn tập sự cố; kiểm thử phân quyền; đánh giá mở rộng; README

Mã sinh viên B24DCCC065 · Lớp RIPT1304-20261-01

Bạn giữ phần "khi hỏng thì sao": sao lưu mỗi ngày, khôi phục thử, đo thời gian gián đoạn và lượng dữ liệu mất, kiểm tra ai được vào đâu, và đề xuất mở rộng lên 10.000, 100.000 sinh viên. Thầy rất hay hỏi phần này vì nó cho thấy nhóm có nghĩ tới sự cố hay không.

Mục lục

Phần 0. Bắt đầu từ con số 0: dòng lệnh, Git, Docker

- 0.1 Cách đọc tài liệu này
- 0.2 Lab là gì, vì sao làm hỏng cũng không sao
- 0.3 Terminal và các lệnh tối thiểu
- 0.4 Cài công cụ (làm 1 lần)
- 0.5 Git: lấy mã về, xem mình đã sửa gì, hoàn tác
- 0.6 Mở và sửa tệp bằng VS Code
- 0.7 Docker: lab nằm ở đâu trên máy bạn

Phần 1. Kiến thức nền cả nhóm phải nắm

- 1.1 Đề bài yêu cầu gì
- 1.2 Hệ thống gồm những phần nào (ví như một quán ăn)
- 1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)
- 1.4 Lab của nhóm trông như thế nào
- 1.5 Repo này có gì, phần của mình ở đâu
- 1.6 Dựng lab trên máy của bạn
- 1.7 Quy tắc chung khi sửa cấu hình
- 1.8 Lỗi hay gặp khi dựng lab và cách xử lý
- 1.9 Kubernetes cho người chưa dùng bao giờ
- 1.10 Đọc một tệp YAML của Kubernetes
- 1.11 Đọc một script bash
- 1.12 Bản đồ thư mục infra: tệp nào để làm gì
- 1.13 Xem cụm đang chạy: các lệnh kubectl cơ bản
- 1.14 Buổi thực hành đầu tiên (khoảng 1 giờ)
- 1.15 Đọc Grafana, Prometheus, Alertmanager
- 1.16 Nối dự án với kiến thức các buổi học

Phần 2. Phần việc của bạn

- 2.1 Phiếu đề giao gì cho bạn
- 2.2 Sản phẩm của bạn trong hồ sơ

Phần 3. Sao lưu và khôi phục từ đầu

- 3.1 Sao lưu là gì, vì sao "sao lưu xong" chưa đủ
- 3.2 RPO và RTO (2 con số quan trọng nhất của phần bạn)
- 3.3 Sao lưu chạy như thế nào trong lab
- 3.4 pg_dump và pg_basebackup (phần mở rộng tự chọn của bạn)
- 3.5 Thử sao lưu khi có lỗi

Phần 4. Kiểm thử 3 và diễn tập sự cố

- 4.1 Kiểm thử 3, phần dừng dịch vụ
- 4.2 Kiểm thử 3, phần mất dữ liệu
- 4.3 Diễn tập sự cố (chaos)
- 4.4 Runbook: khi xảy ra sự cố thì làm gì

Phần 5. Phân quyền và chặn mạng

- 5.1 Ma trận quyền
- 5.2 Phân biệt 401, 403, 404, 410 (hay bị hỏi)
- 5.3 Kiểm thử phân quyền: 9 ca
- 5.4 Chặn mạng trong cụm (NetworkPolicy)

Phần 6. Sửa và cấu hình trực tiếp

- 6.1 Chạy sao lưu ngay và khôi phục ngay
- 6.2 Đổi giờ sao lưu

6.3 Đổi số bản giữ lại
6.4 Đổi ngưỡng cảnh báo
6.5 Diễn tập 1 sự cố trước mặt thầy
6.6 Đổi quyền

Phần 7. Đánh giá mở rộng, chạy lại và số liệu

7.1 Đánh giá mở rộng theo số đo
7.2 Chạy lại phần của bạn, từng bước
7.3 Số liệu để nói khi bảo vệ
7.4 README của hồ sơ: viết gì, vì sao, kiểm thế nào

Phần 8. Câu hỏi vấn đáp

Câu hỏi về phần của bạn
Câu hỏi chung ai cũng phải trả lời được

Phần 9. Việc cần làm trước khi nộp và bảo vệ

Trước khi nộp
Trình bày 3 phút
Trước buổi bảo vệ

> Cách đọc: chưa dùng dòng lệnh, Git, Docker bao giờ thì bắt đầu từ **Phần 0**; sau đó phần 1 (kiến thức chung, Kubernetes, buổi thực hành đầu tiên), rồi lần lượt các phần riêng của bạn. Chỗ nào có **[mở trên GitHub]** là bấm được, mở đúng tệp và đúng dòng trong repo. Đoạn cấu hình trong tài liệu được trích thẳng từ tệp thật, số dòng bên trái là số dòng trong tệp.

Phần 0. Bắt đầu từ con số 0: dòng lệnh, Git, Docker

0.1 Cách đọc tài liệu này

Tài liệu viết cho người **chưa từng** dùng dòng lệnh, Docker hay Kubernetes. Đọc theo thứ tự: **Phần 0** (công cụ, làm 1 lần) → **Phần 1** (kiến thức chung, Kubernetes, buổi thực hành đầu tiên) → các phần riêng của bạn → câu hỏi vấn đáp → trình bày 3 phút. Đừng nhảy cóc: phần sau dùng từ ngữ của phần trước.

Quy ước trong tài liệu:

Bạn thấy	Nghĩa là
Khối nền xám chứa lệnh, ví dụ `make status`	Gõ (hoặc dán) **từng dòng** vào terminal rồi bấm Enter
Chữ sau dấu `#` trên cùng dòng lệnh	Ghi chú giải thích, không cần gõ (gõ vào cũng không sao, máy bỏ qua)
`<tên pod>`, `<tệp>`	Thay bằng giá trị thật, **bỏ cả dấu** `< >`
Khối có số dòng bên trái `12 | ...`	Trích nguyên văn từ tệp thật để **đọc**, không phải lệnh để gõ
[mở trên GitHub]	Bấm được: mở đúng tệp, đúng dòng trong repo
Cột "Thấy gì là đúng"	Kết quả bạn phải thấy. Thấy khác thì xem mục 1.8 (lỗi hay gặp) trước khi hỏi

0.2 Lab là gì, vì sao làm hỏng cũng không sao

Lab là bản thu nhỏ của hệ thống, chạy ngay trên máy bạn bên trong Docker, dùng **dữ liệu giả**: 1.000 tài khoản dạng `loadtest-0001@example.test`, mật khẩu sinh ngẫu nhiên lúc dựng. Lab **không nối gì** với web Hanni thật trên VPS và không có dữ liệu người dùng thật, nên bạn **không thể** làm hỏng web thật khi thực hành.

- Rối hay hỏng: `make down` xoá sạch lab, dựng lại khoảng 5 phút (mục 1.6).
- Lỡ sửa sai cấu hình: `git restore <tệp>` đưa tệp về như lúc mới lấy về (mục 0.5), rồi `make deploy`.
- Các script cố ý gây sự cố (`chaos.sh`, `failover.sh`, `kt3-...`) chỉ tác động lên lab.

3 điều **không làm**: không chạy các script này trên VPS hay máy chủ thật; không đưa mật khẩu, tệp `.env` lên GitHub; không sửa tay tệp trong `ho-so-nop/data/raw/` và `ho-so-nop/evidence/` (thầy dùng để đối chiếu số trong báo cáo).

0.3 Terminal và các lệnh tối thiểu

Terminal (cửa sổ dòng lệnh) là nơi gõ lệnh bằng chữ thay cho bấm chuột. Mở terminal:

Máy	Cách mở
Ubuntu	Ctrl+Alt+T
macOS	Cmd+Space, gõ "Terminal", Enter
Windows	Cài WSL theo mục 0.4, rồi mở ứng dụng **Ubuntu** từ menu Start. Mọi lệnh trong tài liệu gõ ở cửa sổ Ubuntu này, **không** gõ trong PowerShell hay CMD

Dòng chờ lệnh có dạng `ten@may:~/hanni-server/infra$` : phần sau dấu `:` là **thư mục đang đứng** (`~` là thư mục nhà của bạn).

Lệnh	Làm gì
`pwd`	In thư mục đang đứng
`ls` , `ls -la`	Liệt kê tệp trong thư mục (`-la` : cả tệp ẩn, kèm dung lượng, quyền)
`cd infra` , `cd ..` , `cd ~`	Vào thư mục `infra` / lên 1 cấp / về thư mục nhà
`cat <tệp>`	In cả tệp ra màn hình
`less <tệp>`	Xem tệp dài, cuộn bằng mũi tên, bấm `q` để thoát
`head -20 <tệp>` , `tail -20 <tệp>`	20 dòng đầu / 20 dòng cuối
`tail -f <tệp>`	Xem tệp đang được ghi thêm, Ctrl+C để thoát
`grep -n "chữ" <tệp>`	Tìm dòng có "chữ", in kèm số dòng
`clear`	Xoá màn hình cho dễ nhìn

Phím nên nhớ: **Tab** tự điền nốt tên tệp, thư mục (gõ `scr` rồi Tab thành `scripts/`); **mũi tên lên** gọi lại lệnh trước; **Ctrl+C** dừng lệnh đang chạy (luôn an toàn); **Ctrl+Shift+C / Ctrl+Shift+V** chép, dán trong terminal Linux (macOS: Cmd+C / Cmd+V); **Ctrl+Shift+T** mở thêm 1 tab terminal.

Ba điều hay làm người mới bối rối:

1. **Đứng sai thư mục.** `scripts/kt1.sh` là đường dẫn **tính từ thư mục đang đứng**, nên phải `cd hanni-server/infra` trước. Báo `No such file or directory` thì gõ `pwd` xem mình đang ở đâu.
2. **Lệnh im lặng là thành công.** Nhiều lệnh xong việc không in gì. Lỗi thì thường có chữ `error` , `Error` , `failed` , `denied` . Gõ `echo $?` ngay sau lệnh để xem **mã thoát**: `0` là thành công, số khác là lỗi.
3. **Lệnh chạy lâu.** k6 đo 5 phút, `make images` lần đầu vài phút: để yên, đừng đóng cửa sổ. Muốn làm việc khác thì mở tab terminal mới.

Chạy script: gõ tên script rồi các tham số, cách nhau bằng dấu cách, ví dụ `scripts/kt1.sh k6/ket-qua/kt1.txt` (tham số là nơi ghi kết quả).

0.4 Cài công cụ (làm 1 lần)

Cần: **Git** (lấy mã về), **make** (chạy nhóm lệnh viết sẵn), **curl**, **python3**, **Docker** kèm **Docker Compose v2**, và nên có **VS Code** để sửa tệp.

Windows 10/11

1. Mở PowerShell bằng quyền quản trị (chuột phải → Run as administrator), gõ `wsl --install` , khởi động lại máy. Mở **Ubuntu** từ menu Start, đặt tên người dùng và mật khẩu Linux (gõ mật khẩu sẽ không hiện ký tự, bình thường).
2. Cài **Docker Desktop for Windows**. Trong Settings: General → bật "Use the WSL 2 based engine"; Resources → WSL integration → bật cho Ubuntu.
3. Trong cửa sổ Ubuntu: `sudo apt update && sudo apt install -y git make curl python3` .
4. RAM: WSL mặc định chỉ được dùng một nửa RAM của máy. Lab cần khoảng 3–5 GB, nên máy 8 GB rất sát, máy 16 GB thoải mái. Muốn tăng: tạo tệp `.wslconfig` trong thư mục người dùng Windows (`C:\Users\<tên>\`), ghi 2 dòng `[wsl2]` và `memory=6GB` , rồi chạy `wsl --shutdown` trong PowerShell.

5. Luôn lấy mã về **thư mục nhà của Ubuntu** (`cd ~`), không để trong `/mnt/c/...` (ổ C của Windows): chậm hơn nhiều và hay lỗi quyền chạy script.

Ubuntu

1. `sudo apt update && sudo apt install -y git make curl python3`
2. Cài Docker bằng script chính thức: `curl -fsSL https://get.docker.com | sudo sh`, rồi `sudo usermod -aG docker $USER`, **đăng xuất và đăng nhập lại** để dùng Docker không cần `sudo`.

macOS: `xcode-select --install` (có sẵn git, make), cài Docker Desktop for Mac, Settings → Resources → Memory từ 6 GB. Nhóm chưa thử lab trên macOS; gặp lỗi thì nhờ Đăng chạy trên GitHub Actions.

Kiểm tra sau khi cài:

Lệnh	Thấy gì là đúng
`git --version`	`git version 2.` …
`make --version`	`GNU Make 4.` … (macOS hiện 3.81 cũng được)
`docker --version`	`Docker version` …
`docker compose version`	`Docker Compose version v2.` … Nếu báo lỗi mà chỉ có `docker-compose` (có gạch nối) thì là bản cũ, cần cài lại Docker
`docker run --rm hello-world`	Có dòng `Hello from Docker!`
`python3 --version`	`Python 3.` …

0.5 Git: lấy mã về, xem mình đã sửa gì, hoàn tác

Git lưu toàn bộ lịch sử thay đổi của mã. **Repo** là thư mục mã kèm lịch sử. **Nhánh** (branch) là một dòng phát triển riêng: mã lab nằm ở nhánh `feat/scaling` của repo `hanni-server`, web thật dùng nhánh `main` nên lab không ảnh hưởng web thật. **Commit** là 1 lần lưu, có mã như `4ef8a31`.

```
cd ~
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git   # lấy mã lab về thư mục ~/hanni-server
cd hanni-server
git branch --show-current     # phải in: feat/scaling
git log --oneline -3          # 3 lần lưu gần nhất
git pull                      # lấy bản mới nhất khi nhóm báo có cập nhật
```

Hoàn tác là thứ giúp bạn không sợ sửa sai. Sau khi sửa tệp:

```
git status                            # liệt kê các tệp bạn đã sửa (chữ đỏ, "modified")
git diff                              # xem đúng dòng đã đổi: dòng bắt đầu "-" là cũ, "+" là mới; bấm q để thoát
git restore infra/k8s/app/api.yaml    # đưa 1 tệp về như lúc lấy về (mất phần đã sửa trong tệp đó)
git restore .                         # đưa MỌI tệp về như cũ
```

Đưa tệp cấu hình về như cũ xong thì chạy lại `make deploy` để cụm cũng về như cũ. Bạn **không cần** commit hay push gì lên repo `hanni-server`. Repo tài liệu cong-nghe-nen-tang chỉ để đọc: `git clone https://github.com/DANG-PH/cong-nghe-nen-tang.git` hoặc trên GitHub bấm **Code** → **Download ZIP**.

0.6 Mở và sửa tệp bằng VS Code

1. Cài **VS Code**. Trên Windows cài thêm tiện ích **WSL** (của Microsoft), rồi trong cửa sổ Ubuntu: `cd ~/hanni-server && code .` để mở cả thư mục.
2. Cài tiện ích **YAML** (của Red Hat): gạch đỏ ngay chỗ thụt lề sai trong tệp `.yaml`.
3. Phím hay dùng:

Phím	Làm gì
Ctrl+P, gõ `api.yaml`	Mở tệp theo tên
Ctrl+G, gõ `143`	Nhảy tới dòng 143 (số dòng trong tài liệu này khớp số dòng trong tệp)
Ctrl+F / Ctrl+Shift+F	Tìm trong tệp / tìm trong cả thư mục
Ctrl+S	Lưu
Ctrl+Z	Hoàn tác khi chưa lưu (đã lưu thì dùng `git restore`)
Ctrl+`	Mở terminal ngay trong VS Code, đứng sẵn ở thư mục đang mở

1. Sửa YAML: **chỉ đổi giá trị sau dấu `:`**, giữ nguyên số dấu cách đầu dòng, không thêm phím Tab. Lưu xong mới chạy lệnh áp dụng.
2. Thanh bên trái **Source Control** (biểu tượng nhánh cây) liệt kê tệp đã sửa; bấm vào thấy trước và sau, giống `git diff`.

0.7 Docker: lab nằm ở đâu trên máy bạn

```
Máy của bạn (Ubuntu, WSL trên Windows, hoặc macOS)
└── Docker
    ├── container hanni-lab-server-1  = node k3s-server (172.25.0.2), RAM tối đa 2,8 GB
    │       └── Kubernetes chạy các pod: API, Postgres, Redis, Nginx, Grafana...
    └── container hanni-lab-agent-1   = node k3s-agent  (172.25.0.3), RAM tối đa 1,8 GB
            └── các pod khác (Kubernetes tự chia)
```

- `docker ps` liệt kê container đang chạy: thấy đúng 2 container trên. Các pod nằm **bên trong** 2 container này nên `docker ps` không thấy pod; xem pod bằng `scripts/kubectl` (mục 1.13).
- `docker stats` (Ctrl+C để thoát): CPU, RAM 2 node đang dùng.
- `make stop`: tắt 2 container, **giữ** dữ liệu, trả RAM cho máy. `make start`: bật lại, chờ 1–2 phút cho pod lên đủ.
- `make down`: **xoá** 2 container và ổ đĩa của chúng, mất dữ liệu lab, phải dựng lại từ `make up`.
- Tắt Docker Desktop là lab tắt theo. Khởi động lại máy xong: mở Docker Desktop (Ubuntu: `sudo systemctl start docker` nếu Docker chưa tự chạy), rồi `make start` trong `hanni-server/infra`.

Phần 1. Kiến thức nền cả nhóm phải nắm

1.1 Đề bài yêu cầu gì

Đề 06 của môn yêu cầu: **thiết kế một nền tảng học trực tuyến cho 1.000 sinh viên**. "Thiết kế" ở đây không phải vẽ giao diện, mà là trả lời bằng số các câu hỏi của người làm hệ thống:

- 1.000 sinh viên thì **cùng lúc có bao nhiêu người** dùng? Cần bao nhiêu **băng thông** (đường truyền mạng), bao nhiêu **dung lượng lưu trữ**?
- Hệ thống gồm những phần nào, nối với nhau ra sao (**kiến trúc**)?
- Dựng thử được không, **đo** được không: chậm ở đâu, chịu được bao nhiêu người?
- Khi hỏng (máy chết, mất dữ liệu) thì **khôi phục** thế nào, mất bao lâu, mất bao nhiêu dữ liệu?

Phiếu đề chia thành 4 yêu cầu và 3 bài kiểm thử. Nhóm gọi bằng lời thường như sau:

Phần của đề	Nội dung	Ai lo chính
Yêu cầu 1: thiết kế	Thiết kế web, dữ liệu, video, đăng nhập; phân biệt **tổng tài khoản** với **số người dùng cùng lúc**	Đăng
Yêu cầu 2: tính tài nguyên	Tính băng thông, lưu trữ cho ít nhất 2 tình huống, ghi rõ mọi giả định	Đăng
Yêu cầu 3: triển khai	Dựng 1 web học liệu nhỏ có ít nhất 3 tài nguyên (video, PDF, bài nghe); thử 3 mức tải	Thành (web), Đăng (lab)
Yêu cầu 4: đo và đề xuất	Đo thời gian trả lời, số request mỗi giây, lỗi, CPU/RAM; chỉ ra điểm nghẽn; sao lưu; phân quyền	Vân Anh (đo), Đức (sao lưu, quyền)
Kiểm thử 1	Máy khách mở được trang và tải đúng học liệu	Vân Anh, Thành
Kiểm thử 2	Chạy 3 mức tải, ghi số liệu, điều kiện đo lặp lại được	Vân Anh
Kiểm thử 3	Dừng dịch vụ rồi khôi phục; kiểm tra học liệu và thời gian gián đoạn	Đức

Hồ sơ phải nộp (mục "Hồ sơ nộp chung" của phiếu đề): báo cáo nhóm 12–18 trang; sơ đồ và bảng tài nguyên/IP/quyền; cấu hình, script, mã nguồn; README hướng dẫn dựng lại; bảng kiểm thử có giờ chạy; dữ liệu đo gốc; video demo 5–7 phút; báo cáo cá nhân. Tất cả nằm trong thư mục ho-so-nop.

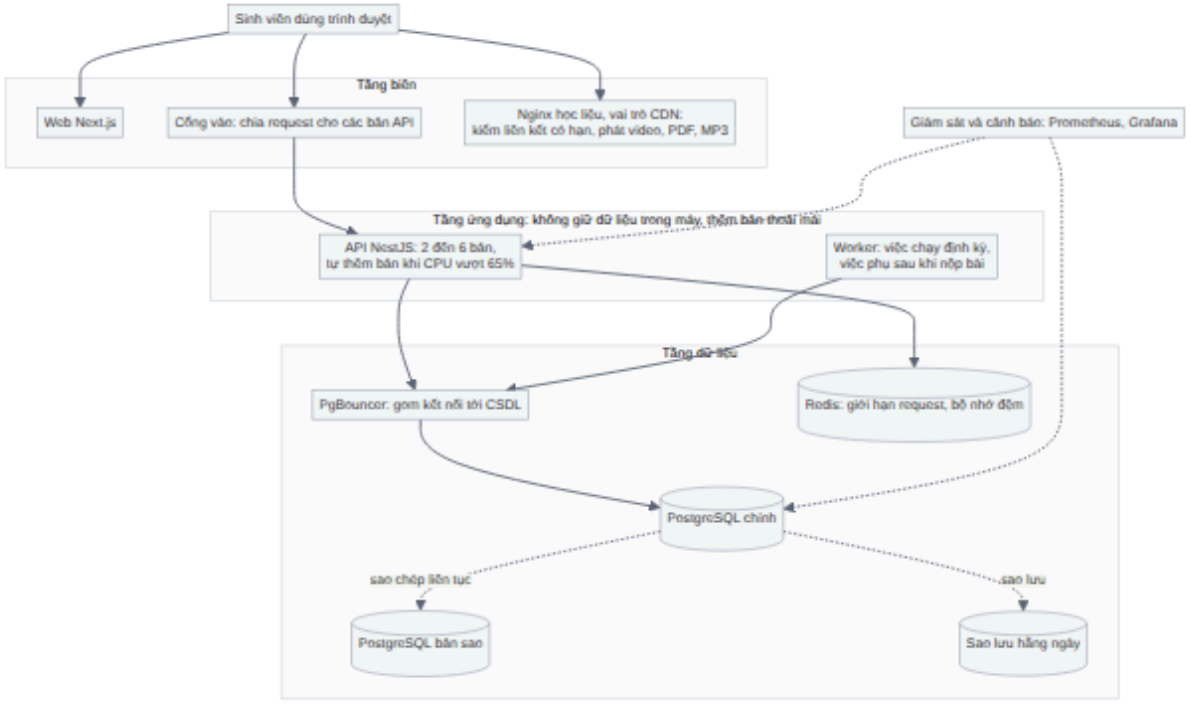
Buổi bảo vệ: mỗi người trình bày phần mình, **mở minh chứng** (tệp kết quả), **làm 1 thao tác hoặc sửa 1 thay đổi nhỏ** ngay trước mặt thầy, rồi **trả lời vấn đáp** (thầy có thể hỏi cả kiến trúc chung). Điểm cá nhân chiếm 50/100: 15 điểm phần kỹ thuật có bằng chứng, 15 điểm giải thích và thao tác trực tiếp, 20 điểm vấn đáp. Tài liệu này giúp bạn làm tốt cả 3 việc đó.

> Hanni chỉ là **hệ thống minh hoạ** (web học tiếng Trung mà Đăng viết từ trước). Thứ được chấm là cách nhóm thiết kế, đo, khôi phục, mở rộng, chứ không phải tính năng học tiếng Trung.

1.2 Hệ thống gồm những phần nào (ví như một quán ăn)

Hãy hình dung hệ thống như một quán ăn đông khách:

Phần trong hệ thống	Ví như	Làm gì
Trình duyệt của sinh viên	Khách hàng	Gửi yêu cầu, nhận kết quả, hiện lên màn hình
Web (Next.js, trang `/hoc-lieu`)	Thực đơn và bàn ăn	Phần giao diện sinh viên nhìn thấy, bấm vào
API (NestJS)	Bếp	Nhận yêu cầu ("cho tôi danh sách học liệu"), xử lý, trả kết quả. Chạy nhiều bản như nhiều đầu bếp
CSDL PostgreSQL	Kho nguyên liệu, sổ sách	Nơi giữ dữ liệu thật: tài khoản, tiến độ học, bài thi. Mất là mất thật
Redis	Bảng ghi chú dán tường	Ghi tạm những thứ cần đọc nhanh, mất cũng dựng lại được (bộ đếm giới hạn, bộ nhớ đệm)
PgBouncer	Người giữ cửa kho	Gom yêu cầu vào kho của nhiều đầu bếp, chỉ cho một số lượng cố định vào cùng lúc để kho không quá tải
Nginx học liệu (vai CDN)	Quầy phát đồ đóng gói sẵn	Phát thẳng video, PDF, MP3 cho khách, không bắt bếp phải làm. Khách phải có "phiếu" (liên kết có hạn)
Worker	Nhân viên hậu cần	Làm việc nền: nhắc học hằng ngày, xử lý việc phụ sau khi nộp bài
Giám sát (Prometheus, Grafana, Alertmanager)	Camera và chuông báo	Đo mọi thứ liên tục, vẽ biểu đồ, kêu khi có sự cố
Sao lưu	Bản photo sổ sách cất két	Mỗi ngày chép CSDL và học liệu ra chỗ khác để hỏng còn lấy lại được



Hình 1. Kiến trúc đề xuất cho 1.000 sinh viên (tệp gốc: ho-so-nop/so-do-va-bang/kien-truc-dich.png)

Ba tầng trong hình: tầng biên (web, cổng vào, Nginx học liệu) là chỗ khách đi vào; tầng ứng dụng (API, worker) là chỗ xử lý; tầng dữ liệu (PostgreSQL, Redis, sao lưu) là chỗ giữ dữ liệu. Ý quan trọng nhất: **tầng ứng dụng không giữ dữ liệu trong máy**, mọi thứ cần nhớ đều nằm ở CSDL hoặc Redis, nên muốn chịu nhiều người hơn chỉ cần **thêm bản API**, không phải sửa code.

1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)

Mạng và web

- **Request (yêu cầu)**: một lần trình duyệt hỏi máy chủ, ví dụ "cho tôi danh sách học liệu". Mỗi lần bấm nút có thể sinh ra vài request.
- **Response (trả lời)**: máy chủ đáp lại request, kèm một **mã trả về** (mã HTTP) cho biết kết quả.
- **API**: tập các "cửa" mà máy chủ mở ra để trình duyệt gọi vào, mỗi cửa là một **route** (đường dẫn), ví dụ `GET /api/resources` (lấy danh sách học liệu), `POST /api/resources/:id/link` (xin liên kết tải). `GET` là đọc, `POST` là gửi hoặc tạo mới.
- **Mã trả về HTTP** hay gặp trong dự án:

Mã	Nghĩa	Gặp ở đâu trong dự án
200	Thành công	Gọi API bình thường
401	Chưa đăng nhập	Khách xin liên kết video cần đăng nhập
403	Đã biết là ai nhưng không có quyền, hoặc chữ ký sai	Sửa chữ ký liên kết học liệu; người học gọi chức năng quản trị
404	Không có thứ này (hoặc giấu đi cho an toàn)	Gọi `/api/metrics` từ ngoài
410	Đã từng có nhưng hết hạn	Dùng liên kết học liệu đã quá 10 phút
429	Gửi quá nhiều, chờ chút rồi thử lại	Một tài khoản gọi quá 300 request trong 1 phút
503	Máy chủ đang quá tải hoặc chưa sẵn sàng, "thử lại sau"	Cơ chế từ chối khi quá tải của nhóm; API mất CSDL
5xx khác (500, 502…)	Lỗi máy chủ thật	Trong các lượt đo chính thức: 0 lỗi loại này

- **Băng thông**: lượng dữ liệu đi qua đường mạng mỗi giây, đơn vị **Mbps** (megabit mỗi giây). Chú ý: 8 bit = 1 byte, nên 8 Mbps = 1 MB mỗi giây. Video 540p của nhóm là 2 Mbps, tức mỗi giây xem tốn khoảng 0,25 MB.

Máy chủ

- **CPU, lõi (core)**: bộ xử lý. "0,25 lõi" nghĩa là được dùng một phần tư sức một lõi. Trong Kubernetes viết `250m` (250 phần nghìn lõi).
- **RAM**: bộ nhớ đang làm việc, viết `256Mi` (256 mebibyte, gần bằng 256 MB). Hết RAM thì máy phải dùng **swap** (lấy ổ đĩa làm RAM tạm), chậm hẳn đi; vì vậy lần đo nào máy bị swap thì nhóm bỏ, đo lại.
- **Tiến trình**: một chương trình đang chạy, ví dụ 1 bản API là 1 tiến trình Node.js.
- **Event loop (vòng xử lý của Node.js)**: Node.js xử lý mọi request bằng một vòng lặp duy nhất. Khi CPU quá bận, vòng này bị **trễ** (event loop lag): request phải xếp hàng chờ dù chưa cần tới CSDL.

Nhóm đo độ trễ này để biết API đang quá tải.

Docker và Kubernetes

- **Docker, container, image**: Docker đóng gói một chương trình cùng mọi thứ nó cần vào một **image** (như hộp cơm đóng sẵn). Chạy image lên thành **container** (hộp cơm đang được ăn). Nhờ vậy máy nào có Docker cũng chạy được giống hệt nhau. Mọi công cụ của lab (k6, ffmpeg, kubectl) đều chạy trong container nên máy không phải cài gì thêm.
- **Kubernetes**: phần mềm **quản lý nhiều container** trên nhiều máy: tự bật lại container chết, tự thêm bản khi đông, chia request đều. **k3s** là bản Kubernetes nhẹ, nhóm chạy nó ngay trong Docker để dựng lab trên 1 máy.
- **Cụm (cluster)**: tập các máy Kubernetes quản lý. **Node**: mỗi máy trong cụm. Lab có 2 node: `k3s-server` (172.25.0.2) và `k3s-agent` (172.25.0.3).
- **Pod**: đơn vị nhỏ nhất Kubernetes chạy, thường là 1 container. 1 bản API = 1 pod.
- **Deployment**: bản mô tả "tôi muốn chạy N pod giống nhau từ image X". Kubernetes giữ đúng N pod, chết cái nào tạo lại cái đó.
- **Bản (replica)**: mỗi pod giống nhau trong một Deployment. "API chạy 6 bản" là 6 pod API cùng phục vụ.
- **Service**: một địa chỉ cố định đứng trước nhiều pod, chia request cho các pod đó. Pod chết, đổi địa chỉ, Service vẫn giữ nguyên.
- **Ingress (cổng vào)**: cửa chính từ ngoài vào cụm, xem đường dẫn để đưa request tới đúng Service (`api.hanni.localhost` tới API, `media.hanni.localhost` tới Nginx học liệu). Lab dùng Traefik làm cổng vào, mở ở `127.0.0.1:18080`.
- **Namespace**: ngăn chia trong cụm. Ứng dụng nằm ở `hanni`, giám sát nằm ở `observability`.
- **Manifest (tệp YAML)**: tệp mô tả thứ muốn chạy (Deployment, Service...). Nhóm sửa manifest rồi **apply** (nạp) vào cụm.
- **kubectl**: lệnh điều khiển cụm. Trong lab gọi qua `scripts/kubectl` (đã nối sẵn vào lab). Vài lệnh hay dùng: `scripts/kubectl -n hanni get pods` (xem các pod), `get hpa` (xem tự mở rộng), `logs <tên pod>` (xem nhật ký).
- **HPA (tự mở rộng)**: Horizontal Pod Autoscaler, cứ 15 giây đo CPU trung bình các bản API; vượt ngưỡng (65% mức CPU yêu cầu) thì thêm bản, thấp lâu thì bớt, trong khoảng 2–6 bản.
- **Probe (kiểm tra sức khoẻ)**: Kubernetes định kỳ gọi thử vào pod. **Readiness** hỏi "sẵn sàng nhận khách chưa?", chưa thì tạm không chia request cho pod đó. **Liveness** hỏi "còn sống không?", nhiều lần không trả lời thì khởi động lại pod.

Dữ liệu

- **CSDL (cơ sở dữ liệu) PostgreSQL**: nơi lưu dữ liệu dạng bảng (bảng `User`, `ReviewLog`...), mỗi dòng là một bản ghi.
- **Kết nối CSDL, pool**: mỗi lần API nói chuyện với CSDL phải dùng một **kết nối**. Pool là "hồ" kết nối mở sẵn để dùng lại. CSDL chỉ chịu được một số kết nối nhất định; hết kết nối thì request phải **xếp hàng chờ**, đây là điểm nghẽn đầu tiên nhóm đo được.
- **Bản sao (replica) CSDL**: một CSDL thứ hai liên tục chép theo CSDL chính, dùng để đọc bớt và để dự phòng khi CSDL chính hỏng.
- **WAL (nhật ký thay đổi)**: PostgreSQL ghi mọi thay đổi vào nhật ký trước. Nhờ nhật ký này mà bản sao chép theo được, và có thể khôi phục về **đúng một thời điểm** trước khi xoá nhầm.

- **Redis**: kho dữ liệu cực nhanh nằm trong RAM, dùng cho thứ mất cũng không sao: bộ đếm giới hạn request, bộ nhớ đệm.
- **Bộ nhớ đệm (cache)**: lưu tạm kết quả đã tính để lần sau trả ngay, không phải tính lại. Nhóm lưu tạm "khung lộ trình học" 60 giây.

Đo hiệu năng

- **Tải**: lượng người và request đổ vào hệ thống. Đo bằng **request/giây** hoặc **lượt/giây** (1 lượt mở bài = 5 request đọc).
- **Người dùng cùng lúc (phiên đồng thời)**: số người đang thật sự dùng ở cùng một thời điểm. 1.000 tài khoản không có nghĩa 1.000 người cùng lúc: tự học rải rác chỉ khoảng **63 người**, cả lớp xem video 300, thi thử đồng loạt 600.
- **Định luật Little**: số người cùng lúc = số người bắt đầu mỗi phút × số phút mỗi người ở lại. Ví như quán ăn: mỗi phút có 2,5 khách vào, mỗi khách ngồi 25 phút, thì lúc nào cũng có khoảng 63 khách trong quán.
- **Thời gian trả lời** (độ trễ): từ lúc gửi request tới lúc nhận trả lời, đơn vị **ms** (mili giây, 1.000 ms = 1 giây).
- **p50, p95, p99 (phân vị)**: xếp thời gian trả lời của mọi request từ nhanh tới chậm. **p95 = 40 ms** nghĩa là 95% request nhanh hơn 40 ms, 5% chậm hơn. Nhóm dùng p95 vì **trung bình** bị số đông nhanh che mất những người phải chờ lâu.
- **Thông lượng**: số request trả lời được mỗi giây.
- **Tỉ lệ lỗi**: phần trăm request bị lỗi.
- **SLO (ngưỡng đạt)**: mức chất lượng nhóm tự đặt ra: **95% lượt đọc dưới 300 ms, 95% lượt ghi dưới 500 ms, lỗi dưới 0,5%**. Đo mà vượt ngưỡng là "không đạt".
- **Điểm gãy**: mức tải cao nhất còn đạt SLO; tăng quá đó là chậm vọt. Lab đo được khoảng 40 lượt/giây (khoảng 200 request/giây).
- **Điểm nghẽn (nút thắt)**: bộ phận bị đầy đầu tiên khiến cả hệ thống chậm. Như con đường: chỗ hẹp nhất quyết định xe chạy được bao nhiêu.
- **Từ chối khi quá tải**: khi quá tải, API trả lời ngay mã 503 "thử lại sau" cho một phần lượt đọc, thay vì nhận hết rồi bắt tất cả chờ 5 giây. Người được phục vụ thì nhanh, người bị từ chối thì biết để thử lại.
- **k6**: công cụ **giả lập nhiều người dùng** gửi request đều đặn rồi đo thời gian trả lời. Nhóm viết **kịch bản** k6 bằng JavaScript.
- **Lab**: môi trường đo riêng của nhóm (cụm k3s 2 node trong Docker, dữ liệu giả 1.000 tài khoản), tách hẳn web thật để đo không ảnh hưởng người dùng.
- **GitHub Actions**: dịch vụ của GitHub cho mượn máy ảo chạy lệnh tự động. Laptop của nhóm không đủ RAM nên **số đo chính thức** chạy trên máy ảo GitHub (4 nhân, 16 GB) ngày 09/11/2026; máy ảo dựng lại lab từ đầu theo README rồi mới đo.

Giám sát và sao lưu

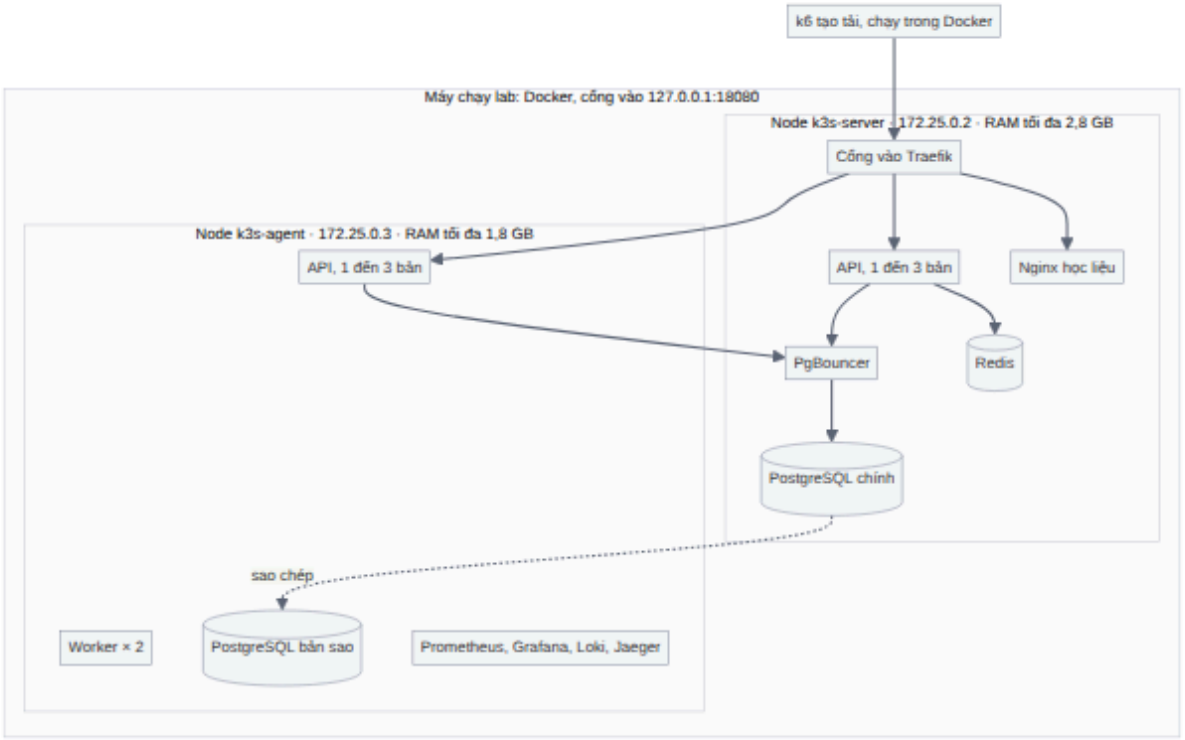
- **Prometheus**: cứ vài giây hỏi số liệu của từng thành phần (API, CSDL, Redis) rồi lưu lại theo thời gian.
- **Grafana**: vẽ số liệu của Prometheus thành biểu đồ. Nhóm có 3 bảng: "Hanni: SLO và tín hiệu vàng", "Hanni: Mở rộng", "Hanni: Dữ liệu" (mục 1.15).
- **Alertmanager**: gửi cảnh báo khi một luật bị vi phạm, ví dụ "từ ngoài gọi API không được quá 30 giây".

- **Blackbox (dò từ ngoài)**: một chương trình đứng ngoài gọi thử API mỗi vài giây như một sinh viên thật, để biết sinh viên có vào được không.
- **Loki, Jaeger**: Loki gom nhật ký (log) của mọi pod; Jaeger lần theo đường đi của một request qua các phần.
- **Sao lưu (backup)**: chép dữ liệu ra một bản riêng để khi hỏng lấy lại được. **Khôi phục (restore)**: đưa bản sao lưu trở lại.
- **RPO**: lượng dữ liệu tối đa có thể mất, tính từ lần sao lưu gần nhất. Sao lưu mỗi ngày thì RPO tối đa 1 ngày.
- **RTO**: thời gian từ lúc hỏng tới lúc dùng lại được. Lab đo được RTO 52 giây khi xoá hẳn CSDL.
- **Mã băm SHA-256**: "dấu vân tay" của tệp, một chuỗi 64 ký tự tính từ nội dung. Sai một byte là chuỗi khác hẳn, nên so mã băm biết tệp tải về hay khôi phục ra có đúng y nguyên không.

Công cụ dựng lab

- **make, Makefile**: `make up` là chạy một nhóm lệnh đã viết sẵn trong tệp `Makefile`. Nhóm có `make up`, `make images`, `make deploy`, `make seed`, `make observability`, `make status`, `make stop`, `make start`, `make down`.
- **YAML**: định dạng các tệp cấu hình. Quy tắc: `khoá: giá trị`, **thụt lề bằng dấu cách** thể hiện cấp (sai thụt lề là hỏng tệp), dòng bắt đầu bằng `#` là ghi chú, chuỗi số nên để trong ngoặc kép như `"600"`.

1.4 Lab của nhóm trông như thế nào



Hình 2. Sơ đồ lab: 2 node, địa chỉ IP và các thành phần (tệp gốc: ho-so-nop/so-do-va-bang/so-do-lab.png)

Thành phần	Địa chỉ	Ghi chú
Node k3s-server	172.25.0.2	RAM tối đa 2,8 GB

Thành phần	Địa chỉ	Ghi chú
Node k3s-agent	172.25.0.3	RAM tối đa 1,8 GB
Cổng vào (ingress)	127.0.0.1:18080	Mọi địa chỉ dưới đây đi qua cổng này
API	http://api.hanni.localhost:18080/api/health	
Grafana	http://grafana.hanni.localhost:18080	Tài khoản `admin`, mật khẩu lấy bằng lệnh ở README
Prometheus	http://prometheus.hanni.localhost:18080	
Alertmanager	http://alertmanager.hanni.localhost:18080	
Web	http://hanni.localhost:3000	Chạy riêng bằng `npx next dev`

1.5 Repo này có gì, phần của mình ở đâu

Đường dẫn	Nội dung
ho-so-nop/bao-cao/	Báo cáo nhóm (.docx, .pdf), 4 báo cáo cá nhân theo mẫu của thầy
ho-so-nop/so-do-va-bang/	2 sơ đồ, bảng tài nguyên/IP/quyền kèm bảng tính tài nguyên
ho-so-nop/cau-hinh-ma-nguon/hanni-server/infra/	**Toàn bộ cấu hình lab**: k8s/ (Kubernetes), k6/ (kịch bản tải), scripts/ (script kiểm thử, sao lưu), observability/ (giám sát), media/ (học liệu)
ho-so-nop/cau-hinh-ma-nguon/hanni-server/src/	Mã API nhóm viết cho đề: học liệu, số liệu đo, giới hạn request, từ chối khi quá tải
ho-so-nop/cau-hinh-ma-nguon/hanni-client/	Trang web học liệu và test tự động
ho-so-nop/data/raw/	Dữ liệu đo gốc, không sửa tay. `gha-09-11/` là lượt đo chính thức
ho-so-nop/evidence/	Minh chứng: ảnh chụp, nhật ký, danh sách mã băm. Danh mục ở `evidence/README.md`
ho-so-nop/bang-kiem-thu.xlsx	Bảng kiểm thử 62 dòng có giờ chạy
ho-so-nop/video-demo.mp4	Video demo 6 phút
Hanni-RIPT1304-Phan-cong.xlsx	Sheet phân công: Tổng quan (vai trò, tiến độ) và Công việc (ai làm gì, minh chứng ở đâu)

Mã nguồn đầy đủ (chạy được) ở 2 repo: hanni-server nhánh feat/scaling và hanni-client nhánh feat/hoc-lieu. Thư mục `cau-hinh-ma-nguon/` chỉ là bản trích các tệp nhóm viết cho đề.

1.6 Dựng lab trên máy của bạn

Cần: đã cài đủ công cụ theo mục 0.4, còn trống khoảng 5 GB RAM và 8 GB đĩa. Mỗi lệnh dưới đây chạy xong mới gõ lệnh tiếp theo.

```
cd ~
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git
cd hanni-server/infra       # MỌI lệnh make và scripts/ đều chạy trong thư mục này
```

Lệnh	Làm gì	Mất bao lâu	Thấy gì là đúng
`make up`	Dựng 3 học liệu mẫu (lần đầu), bật 2 container node, chờ 2 node sẵn sàng, sinh mật khẩu ngẫu nhiên	25 giây; lần đầu lâu hơn vì phải tải image	Bảng 2 node đều `Ready`, các dòng `secret/... created`
`make images`	Build image API từ mã nguồn, nạp vào 2 node	87 giây; lần đầu 5–15 phút	Tự trả lại dấu nhắc, không có chữ `ERROR`
`make deploy`	Nạp mọi tệp trong `k8s/` vào cụm, chờ Postgres, bản sao, PgBouncer sẵn sàng	33 giây	Dòng cuối `deployment "hanni-pgbouncer" successfully rolled out`
`make seed`	Tạo bảng, nạp từ vựng, tạo 1.000 tài khoản giả, khởi động lại API	53 giây	Dòng cuối `deployment "hanni-api" successfully rolled out`
`make observability`	Bật Prometheus, Alertmanager, Grafana, Loki, Jaeger	2 giây, nhưng pod cần thêm 1–2 phút	Các dòng `... created`
`make status`	Xem trạng thái toàn bộ	tức thì	Xem giải thích dưới đây

Thời gian đo trên máy của Đăng ngày 07/11 khi đã có sẵn image (nhật ký đầy đủ: evidence/dung-lai/dung-lai-tu-readme.log). Đoạn cuối của `make up` trên máy đó:

```
kubectl get nodes
NAME         STATUS   ROLES                  AGE   VERSION
k3s-agent    Ready    <none>                 3s    v1.31.6+k3s1
k3s-server   Ready    control-plane,master   13s   v1.31.6+k3s1
./scripts/secrets.sh
secret/hanni-secrets created
secret/grafana-admin created
```

Đầu ra của `make status` (rút gọn) và cách đọc:

```
pod/hanni-postgres-0                       2/2     Running       0          86s
pod/hanni-seed-t94vt                       0/1     Completed     0          48s
pod/hanni-worker-567b6c9597-bb5l8          1/1     Running       0          86s
NAME                                             REFERENCE              TARGETS              MINPODS
MAXPODS   REPLICAS
horizontalpodautoscaler.autoscaling/hanni-api   Deployment/hanni-api   cpu: <unknown>/65%   2
6         2
grafana-5c46b9bc-tdgnx                0/1     ContainerCreating   0          0s
```

- Pod `Running` là đang chạy, `Completed` là việc chạy 1 lần đã xong (tạo bảng, nạp dữ liệu). Cả hai đều đúng.
- HPA ghi `cpu: <unknown>/65%` ngay sau khi dựng là **bình thường**: khoảng 1 phút sau mới có số đo CPU.
- Pod giám sát `ContainerCreating` ngay sau `make observability` cũng bình thường: chờ 1–2 phút rồi gõ lại `make status`, phải thành `Running`.

Kiểm tra API sống: `curl http://api.hanni.localhost:18080/api/health`, thấy `{"status":"ok","db":"up","redis":"up",...}`. Mở trình duyệt vào http://grafana.hanni.localhost:18080 thấy trang đăng nhập Grafana là cổng vào đã thông.

Tạm tắt cho nhẹ máy: `make stop`, bật lại: `make start`, xoá sạch: `make down`. Máy yếu không dựng nổi thì nhờ Đăng chạy trên GitHub Actions (đẩy nhánh lên `do-lab` là máy ảo tự dựng và đo).

1.7 Quy tắc chung khi sửa cấu hình

1. Mở tệp cần sửa trong `hanni-server/infra` (đường dẫn ở bảng "Sửa nhanh" của phần riêng bạn), sửa đúng dòng, giữ nguyên thụt lề.
2. Nạp lại vào cụm: `make deploy`.
3. Nếu đã sửa `k8s/app/config.yaml` hoặc `k8s/media-manifest.json` thì chạy thêm `scripts/kubectl -n hanni rollout restart deployment/hanni-api`, vì API chỉ đọc 2 tệp này lúc khởi động.
4. Nếu đã sửa luật cảnh báo trong `observability/` thì chạy `make observability` rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`.
5. **Kiểm tra lại** bằng lệnh hoặc trang ghi ở cột "Kiểm tra" và nói được kết quả thay đổi thế nào.

> Khi thầy bảo "đổi X rồi chạy lại", hãy nói to 3 ý: (1) X nằm ở tệp nào, dòng nào; (2) đổi xong thì hệ thống sẽ thay đổi gì; (3) lệnh nào kiểm tra được điều đó. Làm như vậy là ăn trọn điểm "giải thích và thao tác trực tiếp".

1.8 Lỗi hay gặp khi dựng lab và cách xử lý

Cách tự tìm lỗi trước khi hỏi: (1) `make status` xem pod nào không ở trạng thái `Running`; (2) `scripts/kubectl -n hanni describe pod <tên pod>`, đọc mục **Events** ở cuối (Kubernetes ghi lý do bằng tiếng Anh, ví dụ thiếu RAM, thiếu image); (3) `scripts/kubectl -n hanni logs <tên pod>` xem nhật ký của chính chương trình.

Hiện tượng	Nguyên nhân	Cách xử lý
`permission denied ... docker.sock` khi chạy `make`	Tài khoản chưa có quyền dùng Docker	`sudo usermod -aG docker $USER`, đăng xuất rồi đăng nhập lại (hoặc chạy lệnh bằng `sudo`)
`port is already allocated` khi `make up`	Cổng 18080 đã có chương trình khác dùng	Tắt chương trình đó, hoặc đổi số 18080 ở dòng `127.0.0.1:18080:80` trong `compose.yaml` (mọi địa chỉ đổi theo)
Lần đầu `make up`, `make images` rất lâu	Phải tải image k3s, PostgreSQL, Node.js…	Bình thường, 10–20 phút lần đầu, các lần sau nhanh
Pod API báo `ErrImageNeverPull`	Chưa chạy `make images`: image API build tại chỗ, manifest đặt `imagePullPolicy: Never` (không tải từ mạng)	`make images` rồi `make deploy`
Pod `Pending` hoặc `OOMKilled`, máy rất chậm	Thiếu RAM: 2 node được dùng tới 2,8 GB + 1,8 GB	Đóng bớt ứng dụng; chỉ thử API thì bỏ `make observability`; máy dưới 8 GB RAM thì nhờ Đăng chạy trên GitHub Actions
`kubectl exec` báo lỗi chứng chỉ sau khi khởi động lại máy	Docker đổi IP của node	`compose.yaml` đã cố định IP; nếu vẫn gặp: `make down` rồi dựng lại

Hiện tượng	Nguyên nhân	Cách xử lý
Đăng nhập trên web xong vẫn như khách	Mở `localhost:3000` thay vì `hanni.localhost:3000`, trình duyệt không gửi cookie sang API	Mở đúng `http://hanni.localhost:3000`
k6 báo vài % lỗi ở đầu lần đo, mức cao hơn lại không lỗi	Đo đúng lúc Kubernetes đang thêm bản API	Chờ đủ bản rồi mới đo (`scripts/cho-du-ban.sh`; `chay-kt2.sh` đã tự làm)
Sau khi chạy `failover.sh` hoặc `do-phan-manh.sh` lab chạy lạ	`failover.sh` cố ý tắt CSDL chính và đưa bản sao lên thay; `do-phan-manh.sh` chia hẳn bảng ReviewLog. Lab không còn như lúc đầu	`make down` rồi dựng lại từ đầu trước khi đo tiếp

1.9 Kubernetes cho người chưa dùng bao giờ

Kubernetes giải quyết việc gì

Không có Kubernetes, muốn chạy API ta mở máy chủ, gõ lệnh chạy chương trình. API chết thì phải có người bật lại; đông khách thì phải tự mở thêm bản và tự chia request cho các bản. Kubernetes làm hộ những việc đó theo một nguyên tắc duy nhất: **bạn viết ra trạng thái mong muốn, Kubernetes liên tục so với trạng thái đang có và tự sửa cho khớp**. Ví dụ bạn viết "cần ít nhất 2 bản API". Một bản chết, đang có 1, khác mong muốn, Kubernetes tự tạo bản mới. Ví như quản lý quán ăn có tờ giấy ghi "ca này cần 2 đầu bếp": 1 người nghỉ thì quản lý gọi người khác vào, không cần chủ quán dặn.

"Trạng thái mong muốn" được viết trong các **tệp YAML** (mục 1.10). Lệnh `make deploy` nạp các tệp đó vào cụm; từ lúc đó Kubernetes tự giữ.

> **Web Hanni thật không chạy trên Kubernetes.** Trên VPS (4 vCPU, 7,8 GB RAM, dùng chung với vài dự án khác, vài người dùng thật), API chạy bằng **pm2** (trình giữ cho chương trình Node.js luôn chạy, chết thì tự bật lại), PostgreSQL, Redis, Nginx chạy bằng **Docker**. Kubernetes chỉ có trong **lab** để thử phương án cho 1.000 sinh viên: tự mở rộng, tự thay bản chết, rải bản ra nhiều máy, chặn mạng. Nếu thầy hỏi "sao không dùng Kubernetes luôn": với 1 máy và vài người dùng thì pm2 + Docker đủ và nhẹ hơn (riêng k3s đã tốn khoảng 650 MB RAM); khi cần nhiều máy, nhiều bản API cho 1.000 sinh viên mới cần tới Kubernetes. Đây chính là "kiến trúc hiện tại" và "kiến trúc đích" trong báo cáo nhóm.

Một request đi qua những gì trong lab

```
Trình duyệt gọi http://api.hanni.localhost:18080/api/resources
   │  cổng 18080 trên máy bạn, Docker chuyển vào node k3s-server (compose.yaml)
   ▼
Ingress (Traefik)          xem tên miền "api.hanni.localhost" → chuyển cho Service hanni-api
(k8s/app/ingress.yaml)
   ▼
Service hanni-api          chọn 1 trong các pod mang nhãn app: hanni-api, chia đều
(k8s/app/api.yaml)
   ▼
Pod API (1 trong 2–6 bản)  chạy NestJS, xử lý request
   ├──► Service hanni-pgbouncer → Pod PgBouncer → Pod hanni-postgres-0 (CSDL)  (k8s/data/)
   └──► Service hanni-redis → Pod Redis                                        (k8s/data/redis.yaml)
```

Video, PDF, MP3 đi đường khác: `media.hanni.localhost` → Ingress → Service `hanni-media` → Pod Nginx (`k8s/app/media.yaml`), không qua API.

Các loại đối tượng trong lab

Mỗi khối trong tệp YAML khai 1 **đối tượng**, dòng `kind:` cho biết loại. Lab dùng các loại sau (đếm trong `k8s/` và `observability/`):

Loại (`kind`)	Ví như	Trong lab	Tệp
Namespace	Khu riêng trong quán	`hanni` (ứng dụng), `observability` (giám sát)	`k8s/namespace.yaml`
Deployment	Tờ giấy "cần N đầu bếp giống nhau"	API, worker, PgBouncer, Redis, Nginx học liệu, Grafana… (15 cái)	`k8s/app/api.yaml` …
ReplicaSet	Ca làm cụ thể theo tờ giấy đó	Deployment tự tạo, bạn không viết. Mỗi lần đổi cấu hình API sinh ReplicaSet mới, bản cũ được thay dần	(tự sinh)
Pod	1 đầu bếp đang làm	Tên dạng `hanni-api-5456c9c9cb-5xzh5` = tên Deployment + mã ReplicaSet + mã riêng của pod	(tự sinh)
StatefulSet	Thủ kho có tên và chìa khoá riêng	Postgres chính và bản sao: tên cố định `hanni-postgres-0` , mỗi pod có ổ đĩa riêng, pod chết tạo lại vẫn gắn đúng ổ cũ	`k8s/data/postgres*.yaml`
PersistentVolumeClaim	Phiếu xin 1 tủ chứa đồ	Ổ đĩa cho Postgres (2 GB), thư mục sao lưu	`k8s/data/` , `k8s/backup/`
Service	Số điện thoại cố định của 1 bộ phận	`hanni-api` , `hanni-pgbouncer` , `hanni-redis` … Pod đổi, Service vẫn giữ tên	trong từng tệp
Ingress	Lễ tân ở cửa	Tên miền nào đi tới Service nào	`k8s/app/ingress.yaml`
ConfigMap	Bảng nội quy dán trong bếp	Biến môi trường của API (`hanni-env`), cấu hình Nginx, danh sách học liệu, script sao lưu	`k8s/app/config.yaml` …
Secret	Két giữ chìa khoá	Mật khẩu CSDL, khoá ký liên kết. Sinh ngẫu nhiên lúc `make up` , không nằm trong tệp nào	`scripts/secrets.sh`
HorizontalPodAutoscaler (HPA)	Quản lý gọi thêm đầu bếp khi đông	Tự đổi số bản API 2–6 theo CPU	`k8s/app/api.yaml`

Loại (`kind`)	Ví như	Trong lab	Tệp
PodDisruptionBudget (PDB)	"Luôn phải có ít nhất 1 người trong bếp"	Khi bảo trì node, luôn còn ít nhất 1 bản API	`k8s/app/api.yaml`
Job, **CronJob**	Việc làm 1 lần / việc làm theo lịch	Job tạo bảng, nạp dữ liệu (`make seed`); CronJob sao lưu 02:00	`k8s/app/jobs.yaml` , `k8s/backup/cronjob.yaml`
NetworkPolicy	Quy định ai được vào phòng nào	Chặn mọi đường mạng trong cụm, chỉ mở đường cần thiết	`k8s/policy/network-policy.yaml`

Tự mở rộng (HPA): làm được gì, không làm được gì

Có, lab **tự mở rộng thật**: HPA trong `k8s/app/api.yaml` cứ 15 giây đo CPU trung bình các bản API, rồi tính số bản cần theo công thức của Kubernetes:

> **Số bản cần = làm tròn lên (số bản hiện có × CPU hiện tại ÷ CPU mục tiêu)**. Mục tiêu là 65% mức CPU yêu cầu (0,25 lõi). Ví dụ 2 bản đang dùng trung bình 130%: 2 × 130 ÷ 65 = 4, Kubernetes nâng lên 4 bản. Kết quả luôn bị giữ trong khoảng `minReplicas` 2 tới `maxReplicas` 6.

Đo thật trong lượt chính thức (tăng tải dần): 4 bản (giây 0) → 3 bản (giây 45) → 5 bản (giây 90) → 6 bản (giây 120). Mỗi lần đổi, Kubernetes ghi 1 **sự kiện** (xem bằng `scripts/kubectl -n hanni get events`), đây là dòng thật của lượt đo:

```
24m  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 2; reason: Current
number of replicas below Spec.MinReplicas
22m  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 3; reason: cpu resource
utilization (percentage of request) above target
8m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 5; reason: All metrics
below target
7m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 4; reason: All metrics
below target
6m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 3; reason: All metrics
below target
5m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 5; reason: cpu
resource utilization (percentage of request) above target
5m14s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 6; reason: cpu
resource utilization (percentage of request) above target
```

Dòng 1: thêm lên 3 bản vì "CPU vượt mục tiêu". Dòng 2: bớt về 5 bản vì "mọi chỉ số dưới mục tiêu" (lúc hết tải).

HPA không làm (thầy hay hỏi để xem bạn có hiểu giới hạn không):

- **Không thêm máy.** HPA chỉ thêm pod trên các node đang có. Thêm máy là việc của bộ tự thêm node (cluster autoscaler) trên cloud, lab không có (2 node chạy chung 1 máy).
- **Không mở rộng CSDL, Redis.** Chỉ API được tự mở rộng vì API không giữ dữ liệu. Postgres có 1 bản chính + 1 bản sao cố định; muốn hơn phải thiết kế lại.
- **Không kịp với đợt dồn 30 giây** (cả lớp nộp bài): đo 15 giây 1 lần, bản mới cần vài chục giây mới sẵn sàng. Trước giờ thi phải tăng sẵn `minReplicas` .

- **Không cứu được khi CSDL là điểm nghẽn**: thêm bản API chỉ làm thêm người xếp hàng chờ CSDL.

1.10 Đọc một tệp YAML của Kubernetes

Mọi đối tượng đều có 4 phần:

Dòng	Ý nghĩa
`apiVersion:`	Phiên bản cách viết của loại đối tượng này (`apps/v1` , `v1` ...). Chép đúng như mẫu, không phải nghĩ
`kind:`	Loại đối tượng (bảng ở mục 1.9)
`metadata:`	Tên (`name`), nhãn (`labels`), ghi chú cho công cụ khác (`annotations`)
`spec:`	**Phần mong muốn**: phần bạn đọc và sửa nhiều nhất

Quy tắc chữ: thụt lề bằng **dấu cách** thể hiện cấp (con nằm trong cha); dòng bắt đầu bằng `-` là 1 phần tử của danh sách; `#` là ghi chú; `---` ngăn 2 đối tượng trong cùng 1 tệp. Muốn biết tệp có những gì: `grep -n "kind:" k8s/app/api.yaml` (in ra Deployment, Service, HPA, PDB).

Nhãn (labels) và bộ chọn (selector): đây là cách các đối tượng tìm thấy nhau. Pod API mang nhãn `app: hanni-api` . Deployment quản lý những pod có nhãn đó, Service gửi request tới những pod có nhãn đó, HPA, PDB, NetworkPolicy cũng chọn theo nhãn đó. Sửa nhãn ở một chỗ mà quên chỗ kia thì Service "không thấy" pod nào, API mất kết nối dù pod vẫn chạy.

Đọc thử phần đầu tệp API (Deployment):

hanni-server/infra/k8s/app/api.yaml, dòng 1–46 (mở trên GitHub):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hanni-api
  labels:
    app: hanni-api
spec:
  # Không khai replicas: HPA giữ số bản, khai ở đây thì mỗi lần apply sẽ kéo số bản về lại giá trị này.
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: hanni-api
  template:
    metadata:
      labels:
        app: hanni-api
        tier: backend
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"
        prometheus.io/path: /api/metrics
    spec:
      terminationGracePeriodSeconds: 40
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: hanni-api
      containers:
        - name: api
          image: hanni-api:lab
          imagePullPolicy: Never
          ports:
            - name: http
              containerPort: 8000
          envFrom:
            - configMapRef:
                name: hanni-env
            - secretRef:
                name: hanni-secrets
```

Dòng	Đọc là
1–6	Một Deployment tên `hanni-api`, gắn nhãn `app: hanni-api`
8	Ghi chú: cố ý **không** khai số bản ở đây, vì HPA giữ số bản; khai ở đây thì mỗi lần `make deploy` số bản bị kéo về
9–13	Cách thay phiên bản mới: thêm tối đa 1 bản mới trước (`maxSurge: 1`), không tắt bản cũ nào khi bản mới chưa sẵn sàng (`maxUnavailable: 0`), nên đổi cấu hình không làm gián đoạn
14–16	Deployment quản lý các pod có nhãn `app: hanni-api`

Dòng	Đọc là
17–25	**Khuôn** để tạo từng pod: nhãn của pod, và ghi chú cho Prometheus biết lấy số liệu ở cổng 8000, đường `/api/metrics`
27	Khi tắt, cho pod 40 giây làm nốt request đang dở
28–34	Rải các bản API đều lên 2 node, để 1 node chết vẫn còn bản ở node kia
35–41	Container `api` chạy image `hanni-api:lab` (build tại chỗ bằng `make images`, nên không tải từ mạng: `Never`), nghe cổng 8000
42–46	Nạp biến môi trường từ ConfigMap `hanni-env` (tệp `k8s/app/config.yaml`) và Secret `hanni-secrets` (mật khẩu)

Phần còn lại của tệp (tài nguyên CPU/RAM, kiểm tra sức khoẻ, Service, HPA, PDB) được giải thích trong tài liệu của Đăng, phần 5.

ConfigMap là nơi đặt cấu hình dạng `TÊN: giá trị`, API đọc thành biến môi trường lúc khởi động (vì vậy sửa xong phải khởi động lại API):

hanni-server/infra/k8s/app/config.yaml, dòng 1–11 (mở trên GitHub):

```
   1 | apiVersion: v1
   2 | kind: ConfigMap
   3 | metadata:
   4 |   name: hanni-env
   5 | data:
   6 |   NODE_ENV: production
   7 |   PORT: "8000"
   8 |   API_PREFIX: /api
   9 |   APP_URL: http://api.hanni.localhost:18080
  10 |   # Client chạy ở hanni.localhost cho cùng site với API, nếu không trình duyệt không nhận
cookie đăng nhập.
  11 |   FRONTEND_URL: http://hanni.localhost:3000
```

Giá trị số hay `true` / `false` phải để trong ngoặc kép (`"8000"`, `"false"`) vì biến môi trường luôn là chữ.

Kustomization là danh sách các tệp mà `make deploy` nạp vào cụm. Tệp mới không có tên ở đây thì không bao giờ được nạp:

hanni-server/infra/k8s/kustomization.yaml, dòng 5–29 (mở trên GitHub):

```
resources:
  - namespace.yaml
  - data/postgres.yaml
  - data/postgres-replica.yaml
  - data/pgbouncer.yaml
  - data/redis.yaml
  - app/config.yaml
  - app/api.yaml
  - app/worker.yaml
  - app/media.yaml
  - app/ingress.yaml
  - policy/network-policy.yaml
  - backup/cronjob.yaml

configMapGenerator:
  - name: hanni-backup-script
    files:
      - backup/backup.sh
  - name: hanni-resources
    files:
      - manifest.json=media-manifest.json

generatorOptions:
  disableNameSuffixHash: true

```

Phần `configMapGenerator` biến `backup.sh` và `media-manifest.json` thành ConfigMap. Đó là lý do sửa danh sách học liệu chỉ cần `make deploy` rồi khởi động lại API, không phải build lại image.

1.11 Đọc một script bash

Các tệp `.sh` trong `scripts/` là **script bash**: một chuỗi lệnh dòng lệnh viết sẵn để khỏi gõ tay. **Mẹo quan trọng nhất**: mọi script của nhóm đều ghi **mục đích và cách gọi ở 2–8 dòng ghi chú đầu tệp**, đọc chỗ đó trước. Nhiều script gọi không kèm tham số sẽ in hướng dẫn (ví dụ `scripts/chaos.sh`).

Đọc thử 1 script ngắn, đủ các cú pháp hay gặp:

hanni-server/infra/scripts/do-gian-doan.sh, dòng 1–25 (mở trên GitHub):

```
  1 | #!/usr/bin/env bash
  2 | # Gọi API mỗi giây, ghi mã trả về vào CSV. Dừng bằng Ctrl+C thì in tổng thời gian gián đoạn
dài nhất.
  3 | #   ./do-gian-doan.sh ket-qua/dung-api.csv
  4 | set -uo pipefail
  5 |
  6 | out=${1:?Cần đường dẫn tệp CSV}
  7 | url=${URL:-http://api.hanni.localhost:18080/api/health/ready}
  8 | mkdir -p "$(dirname "$out")"
  9 | echo 'thoi_diem,ma' > "$out"
 10 |
 11 | summary() {
 12 |   awk -F, 'NR > 1 {
 13 |     if ($2 ≠ 200) { run++; if (run > max) max = run; bad++ } else run = 0
 14 |   } END {
 15 |     printf "%d lượt, %d lượt lỗi, gián đoạn dài nhất %d giây\n", NR - 1, bad, max
 16 |   }' "$out"
 17 |   exit 0
 18 | }
 19 | trap summary INT TERM
 20 |
 21 | while true; do
 22 |   code=$(curl -s -o /dev/null -m 1 -w '%{http_code}' "$url")
 23 |   echo "$(date -u +%FT%T.%3NZ),$code" >> "$out"
 24 |   sleep 1
 25 | done
```

Dòng	Đọc là
1	`#!/usr/bin/env bash` : chạy tệp này bằng bash. Dòng nào cũng có ở đầu script
2–3	Ghi chú: làm gì, gọi thế nào
4	`set -uo pipefail` : dùng biến chưa có giá trị là dừng (`-u`); lệnh nào trong chuỗi `a | b` lỗi thì cả chuỗi tính là lỗi. Script khác còn thêm `-e` : lệnh nào lỗi là dừng ngay
6	`out=${1:?Cần đường dẫn tệp CSV}` : lấy **tham số thứ nhất** (`$1`) bỏ vào biến `out` ; thiếu thì dừng và in câu báo lỗi
7	`url=${URL:-...}` : lấy biến môi trường `URL` nếu có, không thì dùng giá trị sau `:-` (giá trị mặc định)
8–9	Tạo thư mục chứa tệp kết quả; `>` ghi đè tệp bằng dòng tiêu đề CSV
11–18	Một **hàm** tên `summary` : dùng `awk` đếm lượt lỗi và chuỗi lỗi dài nhất trong tệp CSV
19	`trap summary INT TERM` : khi bạn bấm Ctrl+C thì chạy hàm `summary` trước khi thoát
21–25	Vòng lặp mãi: `curl` gọi API, `$(...)` lấy kết quả của lệnh (mã HTTP) bỏ vào biến; `>>` **ghi thêm** 1 dòng vào cuối tệp; nghỉ 1 giây

Bảng cú pháp bash hay gặp trong các script của lab:

Viết	Nghĩa
`$1` , `$2` , `"$@"`	Tham số thứ 1, thứ 2, tất cả tham số khi gọi script
`${X:-mặc định}` , `${X:?báo lỗi}`	Biến X, thiếu thì lấy mặc định / thiếu thì dừng
`$(lệnh)`	Chạy lệnh, lấy chữ nó in ra
`a | b`	Đưa kết quả lệnh a làm đầu vào lệnh b

Viết	Nghĩa
`> tệp`, `>> tệp`, `2>&1`	Ghi đè vào tệp / ghi thêm vào cuối tệp / gộp thông báo lỗi chung với kết quả
`lệnh &`, `wait`	Chạy lệnh ở nền (song song) / chờ lệnh nền xong
`case "$1" in api) … ;; esac`	Chọn nhánh theo tham số (xem `scripts/chaos.sh`)
`for x in …; do …; done`, `while …; do …; done`	Vòng lặp
`[ -f tệp ]`, `[ "$a" = "$b" ]`	Kiểm tra tệp có tồn tại / 2 chuỗi bằng nhau
`exit 1`	Thoát và báo lỗi (0 là thành công, khác 0 là lỗi)

`scripts/kubectl` cũng là 1 script, chỉ 8 dòng: nó chạy lệnh `kubectl` **bên trong** node `server` bằng `docker compose exec`, nên máy bạn không phải cài kubectl và không thể trỏ nhầm sang cụm khác. Vì vậy trong lab luôn gõ `scripts/kubectl …` thay vì `kubectl …`.

Script `.js` trong `k6/` là kịch bản tải viết bằng JavaScript (tài liệu của Vân Anh giải thích); `.py` là Python, đọc ghi chú trong cặp `"""` ở đầu tệp.

1.12 Bản đồ thư mục infra: tệp nào để làm gì

Thư mục cau-hinh-ma-nguon/hanni-server/infra/. Cột cuối là người cần nắm kỹ tệp đó.

Tệp hoặc thư mục	Để làm gì	Ai nắm
`Makefile`	Các lệnh `make up/images/deploy/seed/observability/status/stop/start/down`, mỗi lệnh là 1 nhóm lệnh viết sẵn	Cả nhóm
`compose.yaml`	Bật 2 node k3s bằng Docker: IP cố định, giới hạn RAM, mở cổng 18080	Đăng
`README.md`	Cách dựng, địa chỉ, bảng "Sửa nhanh khi bảo vệ"	Cả nhóm
`k8s/kustomization.yaml`, `k8s/namespace.yaml`	Danh sách tệp `make deploy` nạp; tạo khu `hanni`	Đăng
`k8s/app/api.yaml`	API: Deployment, Service, HPA, PDB	Đăng
`k8s/app/config.yaml`	Biến môi trường của API, có ghi chú từng dòng	Đăng
`k8s/app/worker.yaml`	2 worker: việc định kỳ, outbox	Đăng
`k8s/app/ingress.yaml`	Tên miền nào tới Service nào; giấu `/api/metrics` khỏi bên ngoài	Đăng
`k8s/app/jobs.yaml`	Việc chạy 1 lần lúc `make seed`: tạo bảng, nạp từ vựng, 1.000 tài khoản giả	Đăng
`k8s/app/media.yaml`, `k8s/media-manifest.json`	Nginx phát học liệu, kiểm chữ ký; danh sách 3 học liệu	Thành
`media/make-sample.sh`, `media/nguon/`	Dựng 3 học liệu (video HLS 2 mức, PDF, MP3) từ slide và ghi âm, ghi mã băm	Thành

Tệp hoặc thư mục	Để làm gì	Ai nắm
`k8s/data/postgres.yaml`, `postgres-replica.yaml`	CSDL chính (bật lưu nhật ký WAL) và bản sao đọc	Đăng
`k8s/data/pgbouncer.yaml`, `redis.yaml`	Gom kết nối CSDL; Redis cho giới hạn request, bộ đệm, khoá	Đăng
`k8s/policy/network-policy.yaml`	Chặn mạng trong cụm, chỉ mở đúng đường đi	Đức
`k8s/backup/cronjob.yaml`, `backup.sh`	Lịch và script sao lưu hằng ngày	Đức
`k6/lib.js`	Phần dùng chung của kịch bản tải, có tiêu chí đạt	Vân Anh
`k6/kb1-tu-hoc.js`, `kb2-video.js`, `kb3-thi-thu.js`	3 kịch bản: tự học, cả lớp xem video, thi thử đồng loạt	Vân Anh
`k6/tang-dan.js`, `dang-nhap.js`, `kb1-dong.js`, `ghi-lien-tuc.js`	Tìm điểm gãy; đăng nhập đồng loạt; mô hình đóng; ghi liên tục khi diễn tập	Vân Anh
`k6/thu-tai-nguyen.sh`, `scripts/k6.sh`	Ghi CPU/RAM từng pod khi đo; chạy k6 trong Docker	Vân Anh
`scripts/chay-kt2.sh`, `cho-du-ban.sh`, `chon-muc.py`	Kiểm thử 2; chờ đủ bản API rồi mới đo; chọn 3 mức từ đo thăm dò	Vân Anh
`scripts/kt1.sh`	Kiểm thử 1: tải đủ học liệu như máy khách, so mã băm	Vân Anh, Thành
`scripts/kt3-dung-dich-vu.sh`, `kt3-mat-du-lieu.sh`, `do-gian-doan.sh`	Kiểm thử 3; đo thời gian gián đoạn	Đức
`scripts/chaos.sh`	Diễn tập: tắt 1 bản API, Redis, Postgres, 1 node	Đức
`scripts/restore.sh`, `chep-sao-luu.sh`, `loi-sao-luu.sh`, `so-sanh-sao-luu.sh`	Khôi phục; chép bản sao lưu ra ngoài cụm; thử lỗi sao lưu; so pg_dump với pg_basebackup	Đức
`scripts/kiem-quyen.sh`	Kiểm thử phân quyền	Đức
`scripts/failover.sh`, `doi-chieu-ghi.sh`, `pitr.sh`, `do-phan-manh.sh`, `sql/`	Chuyển sang bản sao; đếm lượt ghi mất; khôi phục về đúng thời điểm; chia bảng	Đăng
`scripts/kubectl`, `secrets.sh`, `cho-du-node.sh`, `xuat-chi-so.py`	kubectl của lab; sinh mật khẩu; chờ đủ 2 node; xuất số Prometheus ra CSV	Đăng

Tệp hoặc thư mục	Để làm gì	Ai nắm
`scripts/so-sanh-video.cjs`, `quay-demo.cjs`, `ci/quay-demo.sh`	So HLS với MP4 trên trình duyệt; quay video demo	Thành
`observability/`	Prometheus, Alertmanager, Grafana (3 bảng ở `dashboards/`), Loki, Alloy, Jaeger, blackbox	Đăng
`observability/rules/hanni-slo.yml`	Luật cảnh báo: khi nào Alertmanager báo	Đức
`otel/tracing.js`	Gắn theo dõi đường đi request để xem trên Jaeger	Đăng
`ci/chung.sh`, `kt2.sh`, `kt3.sh`, `phu.sh`	Các job đo trên GitHub Actions (workflow `.github/workflows/do-lab.yml`)	Đăng

1.13 Xem cụm đang chạy: các lệnh kubectl cơ bản

Chạy trong thư mục `hanni-server/infra`. Thêm `-n hanni` để xem khu ứng dụng, `-n observability` để xem khu giám sát.

Lệnh	Cho biết gì
`scripts/kubectl get nodes`	2 node, trạng thái `Ready`
`scripts/kubectl -n hanni get pods -o wide`	Mọi pod, trạng thái, nằm trên node nào
`scripts/kubectl -n hanni get deploy,statefulset,svc`	Các Deployment, StatefulSet, Service
`scripts/kubectl -n hanni get hpa`	Số bản API hiện tại, CPU hiện tại so với mục tiêu 65%
`scripts/kubectl -n hanni describe pod <tên>`	Mọi chi tiết của 1 pod; mục **Events** cuối cùng cho biết vì sao pod lỗi
`scripts/kubectl -n hanni logs <tên pod>`	Nhật ký của chương trình trong pod (thêm `-f` để xem liên tục)
`scripts/kubectl -n hanni get events --sort-by=.lastTimestamp`	Mọi sự kiện gần đây: tạo pod, thêm bớt bản, lỗi
`scripts/kubectl top pods -n hanni`	CPU (mili lõi, `250m` = 0,25 lõi), RAM từng pod lúc này
`scripts/kubectl -n hanni rollout restart deployment/hanni-api`	Khởi động lại API lần lượt từng bản, không gián đoạn
`scripts/kubectl -n hanni scale deployment hanni-api --replicas=0`	Đổi số bản bằng tay (0 là dừng hẳn, dùng trong Kiểm thử 3)

Ví dụ đầu ra thật của `get pods -o wide` cuối lượt đo 09/11 (đã bớt cột, tệp gốc `data/raw/gha-09-11/ket-qua-kt3/pods-cuoi.txt`):

```
NAME                                  READY  STATUS     RESTARTS  AGE    NODE
hanni-api-5456c9c9cb-5xzh5            1/1    Running    0         6m8s   k3s-agent
hanni-api-5456c9c9cb-b9sw4            1/1    Running    0         6m8s   k3s-server
hanni-migrate-zbbm6                   0/1    Completed  0         32m    k3s-agent
hanni-postgres-replica-0              2/2    Running    0         32m    k3s-server
hanni-worker-7fbffb6b7c-fcm85         1/1    Running    0         6m8s   k3s-server
hanni-worker-7fbffb6b7c-gxvkh         1/1    Running    0         6m8s   k3s-agent
```

- `READY 1/1`: pod có 1 container, đã sẵn sàng. `2/2`: pod có 2 container, ví dụ bản sao Postgres chạy kèm 1 container đưa số liệu cho Prometheus.
- `STATUS Running`: đang chạy. `Completed`: Job đã làm xong (ở đây `hanni-migrate` tạo bảng lúc dựng lab). Gặp `Pending`, `CrashLoopBackOff`, `ErrImageNeverPull` thì xem mục 1.8.
- `RESTARTS`: số lần bị khởi động lại; tăng dần là có vấn đề, xem `logs`.
- `NODE`: pod nằm ở node nào; 2 bản API ở 2 node khác nhau nhờ dòng 28–34 của `api.yaml`.
- Không có `hanni-postgres-0` vì tệp này chụp **sau** diễn tập chuyển sang bản sao: CSDL chính cũ đã bị tắt, `hanni-postgres-replica-0` đang làm CSDL chính.

1.14 Buổi thực hành đầu tiên (khoảng 1 giờ)

Làm hết buổi này một lần là bạn đã tự tay: dựng lab, xem cụm, xem Grafana, tạo tải, thấy tự mở rộng, sửa 1 cấu hình rồi hoàn tác, gây 1 sự cố và thấy hệ thống tự hồi phục. Mọi lệnh chạy trong `~/hanni-server/infra`.

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
1. Dựng lab	Theo mục 1.6	`make status`: mọi pod `Running` hoặc `Completed`	Mục 1.8
2. Xem 2 node	`scripts/kubectl get nodes`	2 dòng `k3s-server`, `k3s-agent` đều `Ready`	`make start`, chờ 1 phút
3. Xem các bản API	`scripts/kubectl -n hanni get pods -l app=hanni-api -o wide`	2 pod `Running`, cột NODE mỗi pod 1 node	Chờ 1 phút, gõ lại
4. Xem tự mở rộng	`scripts/kubectl -n hanni get hpa`	`MINPODS 2`, `MAXPODS 6`, `REPLICAS 2`, `TARGETS` dạng `cpu: 4%/65%`	`<unknown>` thì chờ 1 phút
5. Lấy mật khẩu Grafana	`scripts/kubectl -n observability get secret grafana-admin -o jsonpath='{.data.password}' | base64 -d; echo`	1 chuỗi ký tự ngẫu nhiên	Chưa `make observability` thì chạy
6. Mở Grafana	Trình duyệt: http://grafana.hanni.localhost:18080, đăng nhập `admin` + mật khẩu bước 5, vào http://grafana.hanni.localhost:18080/d/hanni-scaling	Bảng "Hanni: Mở rộng", ô "Số bản API" = 2	Mục 1.15

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
7. Tạo tải	Mở tab terminal thứ 2: `scripts/k6.sh kb1-tu-hoc.js -e RATE=20 -e DURATION=3m`	Trên Grafana sau 1–2 phút: CPU tăng, "Số bản API" có thể lên 3–4; k6 chạy xong in bảng tóm tắt (xem dưới)	Máy yếu thì giảm `RATE=8`
8. Sửa 1 cấu hình	VS Code mở `infra/k8s/app/api.yaml`, Ctrl+G dòng 143, đổi `maxReplicas: 6` thành `4`, Ctrl+S; rồi `make deploy`	`scripts/kubectl -n hanni get hpa` hiện `MAXPODS 4`	Mục 0.6, sửa đúng thụt lề
9. Xem mình đã sửa gì	`git diff`	1 dòng `- maxReplicas: 6` và 1 dòng `+ maxReplicas: 4`	
10. Hoàn tác	`git restore k8s/app/api.yaml` rồi `make deploy`	`get hpa` hiện lại `MAXPODS 6`, `git status` không còn tệp đỏ	
11. Gây sự cố	`scripts/chaos.sh api` (xoá đột ngột 1 bản API)	Gõ ngay lệnh bước 3: 1 pod mới có AGE vài giây, vẫn đủ 2 bản	
12. Tắt lab	`make stop`	Dấu nhắc trở lại; `docker ps` không còn 2 container	

Bảng tóm tắt k6 in ra cuối bước 7 có dạng như sau (đây là dòng thật của 1 lần đo chính thức, mức 36 lượt/giây; bước 7 của bạn tải nhẹ hơn nên số khác):

```
     ✓ lộ trình 200
     checks.....................: 100.00% 54005 out of 54005
     ✓ { kind:read }.............: avg=15.28ms  min=2.81ms  med=8.96ms   max=302.96ms
p(90)=30.74ms  p(95)=41.47ms  p(99)=79.8ms
   ✓ http_req_failed.............: 0.00%   0 out of 54006
     iterations.....................: 10801   35.887685/s
```

Đọc nhanh: dấu `✓` là đạt, `✗` là không đạt; `checks 100%` là mọi request trả đúng mã; `p(95)=41.47ms` là 95% lượt đọc nhanh hơn 41 ms (ngưỡng đạt là 300 ms); `http_req_failed 0.00%` là không lỗi; `iterations` là số lượt đã chạy và số lượt mỗi giây. Chi tiết ở tài liệu của Vân Anh.

> Bạn vừa tự tay làm đúng những việc thầy có thể yêu cầu khi bảo vệ: mở minh chứng (Grafana), đổi 1 thông số rồi giải thích (bước 8–10), cho thấy hệ thống tự hồi phục (bước 11). Làm lại buổi này 2 lần là đủ tự tin.

1.15 Đọc Grafana, Prometheus, Alertmanager

Ba trang web của phần giám sát, đều mở qua cổng 18080 (lab phải đang chạy và đã `make observability`):

Trang	Địa chỉ	Dùng để
Grafana	http://grafana.hanni.localhost:18080	Xem biểu đồ. Tài khoản `admin`, mật khẩu lấy ở bước 5 mục 1.14
Prometheus	http://prometheus.hanni.localhost:18080	Nơi lưu số đo. Menu Status → Targets: mọi nguồn số liệu phải `UP`
Alertmanager	http://alertmanager.hanni.localhost:18080	Danh sách cảnh báo **đang bật**. Trống là không có sự cố

Grafana (giao diện tiếng Anh): menu trái **Dashboards** liệt kê 3 bảng của nhóm, hoặc gõ thẳng địa chỉ. Góc trên phải có ô chọn khoảng thời gian (mặc định vài phút gần nhất) và nút làm mới. Rê chuột lên biểu đồ để xem số tại từng thời điểm.

Bảng	Địa chỉ	Các ô quan trọng
Hanni: SLO và tín hiệu vàng	`/d/hanni-slo`	Độ sẵn sàng 1 giờ (mục tiêu 99,5%), Đọc p95 (dưới 300 ms), Ghi p95 (dưới 500 ms), Lưu lượng, Tỉ lệ lỗi 5 phút, Cảnh báo đang bật, Dò từ ngoài qua ingress (1 là vào được, vẫn đo được khi mọi bản API đã chết), 10 route chậm nhất
Hanni: Mở rộng	`/d/hanni-scaling`	Số bản API, Trần HPA, Request chờ kết nối CSDL, Event loop trễ, Kết nối Postgres (trần 60), CPU từng bản API so với yêu cầu 250 mCPU (vượt 65% là HPA thêm bản), CPU bị bóp do chạm giới hạn, RAM từng pod
Hanni: Dữ liệu	`/d/hanni-du-lieu`	Postgres, Redis có sống không, Dung lượng CSDL, Sao lưu gần nhất (cảnh báo khi quá 26 giờ), Bản sao trễ, Outbox chưa xử lý, Giao dịch mỗi giây

Cảnh báo (luật ở `observability/rules/hanni-slo.yml`): mỗi luật có điều kiện và thời gian phải kéo dài (`for`) mới báo, để không báo nhầm vì 1 giây chập chờn.

Tên cảnh báo	Báo khi
HanniEdgeDown	Dò từ ngoài không gọi được API, kéo dài 30 giây (cảnh báo dùng trong Kiểm thử 3)
HanniApiDown	Không còn bản API nào trả lời, 1 phút
HanniPostgresDown	Không kết nối được PostgreSQL, 1 phút
HanniReadLatencyHigh, HanniWriteLatencyHigh	Đọc p95 vượt 300 ms / ghi p95 vượt 500 ms, 5 phút
HanniErrorBudgetBurnFast, …Slow	Tỉ lệ lỗi 5xx cao tới mức giữ nhịp này thì hết "ngân sách lỗi" của tháng trong 2 ngày / 5 ngày
HanniHpaAtMax	API đã chạm trần 6 bản, 5 phút: tự mở rộng hết sức
HanniDbPoolWaiting	Có request xếp hàng chờ kết nối CSDL, 2 phút
HanniEventLoopLag	Node.js trễ quá 200 ms, 5 phút: API nghẽn CPU
HanniReplicationLagHigh	Bản sao đọc trễ quá 30 giây
HanniOutboxBacklog	Hơn 500 việc phụ sau nộp bài chưa xử lý, 5 phút
HanniBackupStale	Hơn 26 giờ chưa có bản sao lưu thành công

Lab chưa gắn kênh gửi thật (email, Telegram), nên cảnh báo chỉ hiện trên Alertmanager và ô "Cảnh báo đang bật" của Grafana.

1.16 Nối dự án với kiến thức các buổi học

Phiếu đề không bắt buộc phần này, nhưng mẫu báo cáo cá nhân có dòng "Kiến thức học phần đã vận dụng", và thầy ghi rõ ở slide cuối 2 buổi: **buổi 7** "đưa vào bài tập lớn: sơ đồ triển khai + trách nhiệm quản lý + rủi ro"; **buổi 8** "đưa vào bài tập lớn: một ma trận quyền + một kịch bản kiểm thử an toàn". Nhóm đã có đủ: sơ đồ (so-do-va-bang/), bảng ai quản lý phần nào và rủi ro (báo cáo nhóm mục 3.5), ma trận quyền (mục 4.3), kiểm thử phân quyền (mục 5.5).

Bảng dưới lấy theo **slide của thầy** (số slide là số in ở chân slide; tệp slide ở thư mục bài giảng thầy gửi). Mỗi người nắm chắc các dòng có tên mình.

Buổi, slide	Slide nói gì	Trong dự án là	Ai nói
Buổi 1, slide 7, 24	Ứng dụng chỉ là phần nổi; bên dưới là phần cứng, hệ điều hành, tệp, mạng, Cloud, bảo mật, quản trị. Case study bán hàng online	Lab có đủ các tầng: CPU/RAM giới hạn từng pod, Linux trong container, ổ đĩa cho Postgres, mạng và cổng vào, Docker và máy ảo GitHub, phân quyền, giám sát, sao lưu	Đăng
Buổi 2, slide 6, 18	Tiến trình là chương trình đang chạy; luồng nằm trong tiến trình; tiến trình tách biệt hơn, luồng nhẹ hơn	Mỗi bản API là 1 **tiến trình** Node.js, xử lý request bằng 1 luồng chính. Chịu thêm người bằng cách chạy thêm tiến trình (thêm bản API, 2 → 6), bản này lỗi không kéo bản khác chết theo	Đăng
Buổi 2, slide 22–25	Race condition (ví dụ 2 người cùng rút tiền), vùng găng, loại trừ lẫn nhau; tình huống "1.000 người cùng bấm đặt hàng trong 1 phút"	Đúng bài toán **cả lớp nộp bài thi thử**: bấm nộp 2 lần, 2 worker cùng chạy nhắc học. Nhóm dùng khoá chống trùng (CSDL chỉ nhận 1 dòng), hợp đồng thuê (chỉ bản giữ khoá được chạy), `FOR UPDATE SKIP LOCKED` (mỗi việc 1 worker lấy). Đo: 600 lượt nộp có gửi lại, số dòng đúng bằng số lượt	Đăng, Vân Anh
Buổi 3, slide 10–11	Thời gian chờ, thời gian phản hồi, thông lượng, công bằng là các tiêu chí khác nhau	Nhóm đo đúng các thứ đó: **p95** là thời gian phản hồi của 5% người chậm nhất, **request/giây** là thông lượng. Dùng p95 thay vì trung bình để không bỏ quên người phải chờ lâu (ý công bằng)	Vân Anh
Buổi 3, slide 30, 34, 44	Ưu tiên: backup chạy nền ưu tiên thấp, cảnh báo ưu tiên cao; chia lượt (quantum); case study "1.000 request đặt hàng trong 1 phút"	Sao lưu chạy lúc 02:00 khi ít người dùng; khi quá tải, API từ chối bớt lượt **đọc** nhưng **không** từ chối lượt ghi như nộp bài (ưu tiên việc quan trọng). Kubernetes chia CPU cho pod theo `requests` / `limits`, dùng hết phần được phép thì phải chờ (ô Grafana "CPU bị bóp")	Đăng, Đức
Buổi 4, slide 8, 25–26	b nhỏ là bit, B lớn là byte (80 Mbps không bằng 80 MB/s); mạng nhanh phải đo băng thông, độ trễ, mất gói; bài tính 100 MB qua 20 Mbps	**Bảng tính tài nguyên** tính băng thông video theo Mbps (2 Mbps 540p ≈ 0,25 MB mỗi giây); cả lớp xem video cần khoảng 888 Mbps. Đo thật bằng HAR, Lighthouse, so sánh HLS với MP4 trên mạng chậm	Đăng, Thành

Buổi, slide	Slide nói gì	Trong dự án là	Ai nói
Buổi 4, slide 15, 24, 31–34	Thông điệp dài chia thành gói nhỏ có phần đầu; client–server; tên miền và DNS; một lần mở web đi qua những bước nào	Video **HLS** chia thành đoạn 6 giây, trình phát tải từng đoạn. Trình duyệt là client, API và Nginx là server. Cổng vào chọn dịch vụ theo **tên miền** (`api.hanni.localhost`, `media.hanni.localhost`)	Thành
Buổi 4, slide 40	Chậm có thể ở máy, ở mạng hoặc ở dịch vụ; không phải cứ chậm là nâng CPU	Điểm nghẽn đầu tiên nhóm đo được là **hàng chờ kết nối CSDL**, không phải CPU của API; sửa đúng chỗ đó mới tăng được mức tải	Đăng, Vân Anh
Buổi 5, slide 9	Move trong cùng ổ chủ yếu cập nhật thông tin quản lý tệp, không chép lại dữ liệu	Script sao lưu ghi ra **tệp tạm** rồi mới đổi tên thành tệp chính thức: đổi tên gần như tức thì, nên không bao giờ có bản dở dang mang tên bản tốt	Đức
Buổi 5, slide 13–14	Quyền owner/group/others; **journaling không thay thế backup**; 3-2-1 (3 bản, 2 loại phương tiện, 1 bản ngoài vị trí chính)	Nhật ký WAL của PostgreSQL giống journaling, chỉ giữ CSDL nhất quán; nhóm vẫn sao lưu `pg_dump` hằng ngày và đã **thử khôi phục**. Chép bản sao lưu ra ngoài cụm ứng với "1 bản ngoài vị trí chính"; lab chưa có "2 loại phương tiện", đây là giới hạn phải nói	Đức
Buổi 6, slide 8, 13	VLAN không thay ACL, firewall; thiết bị L3 quyết định đường đi, ACL quyết định có cho phép hay không	Các pod chung 1 mạng nhưng **NetworkPolicy** đóng vai ACL: mặc định chặn, chỉ mở đường cần. Ca kiểm thử "API nối thẳng Postgres" bị chặn dù có đường, đúng ý slide 13	Đức
Buổi 6, slide 4, 19	Mạng phân tầng để dễ quản trị, mở rộng, xử lý sự cố; tối ưu phải **đo trước → khoanh vùng → thay đổi → đo lại**	Kiến trúc 3 tầng (biên, ứng dụng, dữ liệu) cùng lý do. Cải tiến làm đúng quy trình slide 19: đo tăng dần, tìm điểm nghẽn trên Grafana, sửa 1 chỗ, đo lại cùng điều kiện (157 → 190 request/giây)	Đăng, Vân Anh
Buổi 7, slide 5, 7, 9	Tăng gấp đôi vCPU chưa chắc nhanh gấp đôi; 3 VM trên 1 ổ chưa phải 3 bản dự phòng độc lập; phải chừa tài nguyên cho host	Thêm bản API không giúp khi CSDL nghẽn. 2 node lab chạy chung 1 máy nên "mất 1 node" chưa chứng minh chịu hỏng phần cứng thật. Mỗi node bị giới hạn RAM (2,8 + 1,8 GB) để chừa phần cho máy	Đăng, Đức
Buổi 7, slide 13–14	Snapshot khác backup; VM có hệ điều hành khách riêng, container dùng chung kernel	Lab dùng **container**: 2 node k3s là 2 container dùng chung kernel, bật trong vài giây. Máy đo GitHub là **VM**. Trên Windows, Docker chạy container Linux qua WSL 2 (1 VM Linux nhẹ), không dùng kernel Windows	Đăng
Buổi 7, slide 17, 22, 24	IaaS/PaaS/SaaS: ai quản lý phần nào; scale up khác scale out ("thêm máy mà dữ liệu phiên chỉ ở 1 máy thì xem lại gì?"); tính chi phí cả vòng đời, gồm dữ liệu truyền ra	**Scale out**: API không giữ dữ liệu trong máy nên thêm bản là chạy, đúng câu hỏi slide 22. VPS thuê là IaaS, web trên Vercel là PaaS, GitHub là SaaS (bảng ở báo cáo nhóm mục 3.5). Băng thông video là chi phí lớn nhất nên giao cho CDN	Đăng, Thành

Buổi, slide	Slide nói gì	Trong dự án là	Ai nói
Buổi 8, slide 4, 8–9	CIA; nhận diện, xác thực, phân quyền là 3 việc khác nhau; miền bảo vệ, quyền tối thiểu	**Bí mật**: video cần đăng nhập, mật khẩu trong Secret. **Toàn vẹn**: mã băm học liệu, chữ ký liên kết. **Sẵn sàng**: nhiều bản API, sao lưu, cảnh báo. Xác thực sai trả 401, phân quyền sai trả 403. Ma trận quyền 4 vai trò, khách chỉ được phần công khai	Đức, Thành
Buổi 8, slide 15–16, 22–23	Phòng vệ nhiều lớp (tài khoản, máy, mạng, dữ liệu, phát hiện và phục hồi); firewall chỉ lọc kết nối; SHA-256 để phát hiện tệp bị đổi; phân quyền, mã hoá, hash, backup là 4 việc khác nhau	Đủ 5 lớp: giới hạn request theo tài khoản; cổng vào giấu `/api/metrics`; chặn mạng trong cụm; ký liên kết và phân quyền API; cảnh báo, sao lưu, thử khôi phục. Kiểm thử 1 so **SHA-256** 30 tệp đúng như demo slide 22	Đức, Thành
Buổi 8, slide 14	"Website chậm đã đủ kết luận bị tấn công từ chối dịch vụ chưa?"	Không: lúc đo tải, web chậm vì hàng chờ CSDL. Nhóm phân biệt bằng số đo trên Grafana chứ không đoán	Vân Anh

Phần 2. Phần việc của bạn

2.1 Phiếu đề giao gì cho bạn

Phiếu đề ghi Sinh viên 4 lo **"Sao lưu, phục hồi và đánh giá mở rộng"**, kèm Kiểm thử 3: dừng dịch vụ rồi khôi phục, kiểm tra học liệu và thời gian gián đoạn.

Vai trò	Bạn làm gì	Phục vụ phần nào của đề
SRE (kỹ sư độ tin cậy)	Sao lưu tự động, khôi phục thử, diễn tập sự cố, đo RPO/RTO	Kiểm thử 3, Yêu cầu 4 (sao lưu)
Kiểm thử phân quyền	9 ca kiểm tra ai được làm gì, chặn mạng trong cụm	Yêu cầu 4 (kiểm soát truy cập)
Đánh giá mở rộng	Đề xuất cho 100 / 1.000 / 10.000 / 100.000 sinh viên theo điểm nghẽn đã đo	Yêu cầu 4
Technical Writer	README dựng lại hệ thống, chạy kiểm thử	Hồ sơ nộp

> "SRE" (Site Reliability Engineering) là người lo cho hệ thống **chạy ổn định**: đo, cảnh báo, khôi phục, tập xử lý sự cố. Không cần viết nhiều code, nhưng phải hiểu hệ thống hỏng kiểu gì và lấy lại thế nào.

2.2 Sản phẩm của bạn trong hồ sơ

Sản phẩm	Ở đâu
Lịch và script sao lưu	k8s/backup/cronjob.yaml, k8s/backup/backup.sh
Script khôi phục, chép ra ngoài, thử lỗi, so sánh	scripts/restore.sh, chep-sao-luu.sh, loi-sao-luu.sh, so-sanh-sao-luu.sh
Kiểm thử 3	kt3-dung-dich-vu.sh, kt3-mat-du-lieu.sh, do-gian-doan.sh
Diễn tập sự cố	chaos.sh
Kiểm thử phân quyền, chặn mạng	kiem-quyen.sh, k8s/policy/network-policy.yaml
Luật cảnh báo	observability/rules/hanni-slo.yml
Kết quả	data/raw/gha-09-11/ket-qua-kt3/, data/rpo-rto.xlsx
Ma trận quyền	so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx, tab "Phân quyền"
README	ho-so-nop/README.md
Báo cáo cá nhân	RIPT1304-20261-01_NhomXX_06_B24DCCC065_BaoCaoCaNhan.docx

Phần 3. Sao lưu và khôi phục từ đầu

3.1 Sao lưu là gì, vì sao "sao lưu xong" chưa đủ

Sao lưu là chép dữ liệu ra một bản riêng để khi hỏng (xoá nhầm, hỏng ổ, bị tấn công) còn lấy lại được. Ba nguyên tắc:

- **Phải ở nơi khác**: để bản sao chung ổ với bản chính thì hỏng ổ là mất cả hai. Nguyên tắc **3-2-1**: 3 bản, trên 2 loại nơi lưu, 1 bản ở nơi khác hẳn.
- **Phải giữ nhiều mốc thời gian**: xoá nhầm hôm qua mà chỉ còn bản sao lưu hôm nay (đã mất dữ liệu) thì vô dụng. Nhóm giữ 14 bản (14 ngày).
- **Phải khôi phục thử**: sao lưu chưa từng khôi phục thì chưa thể tin. Nhóm khôi phục thật, so số dòng và mã băm, đo thời gian.

> Đồng bộ thư mục (như Google Drive) **không phải** sao lưu: xoá nhầm ở máy thì bản trên mây cũng bị xoá theo.

3.2 RPO và RTO (2 con số quan trọng nhất của phần bạn)

- **RPO** (mất tối đa bao nhiêu dữ liệu): tính từ lần sao lưu gần nhất tới lúc hỏng. Sao lưu 2 giờ sáng mỗi ngày, 10 giờ tối hỏng thì mất 20 giờ dữ liệu. RPO của bản sao lưu hằng ngày **tối đa khoảng 1 ngày**.
- **RTO** (mất bao lâu mới dùng lại được): tính từ lúc hỏng tới lúc hệ thống chạy lại đúng. Lab đo được **52 giây** khi xoá hẳn CSDL.

Muốn RPO nhỏ hơn 1 ngày thì cần **nhật ký thay đổi (WAL)**: PostgreSQL ghi mọi thay đổi vào nhật ký, nhóm lưu nhật ký này liên tục (tối đa 60 giây một lần), nên khôi phục được về **đúng thời điểm** trước khi xoá nhầm, chỉ mất tối đa khoảng 1 phút. Phần này Đăng làm sâu; bạn chỉ cần hiểu ý.

3.3 Sao lưu chạy như thế nào trong lab

Lịch chạy: một **CronJob** (việc chạy theo lịch của Kubernetes) tên `hanni-backup`.

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 16–21 (mở trên GitHub):

```
  16 │   # 19:00 UTC là 02:00 giờ Việt Nam. Giờ chạy trên chính cụm nên chính xác, không trễ như lịch
của GitHub Actions.
  17 │   schedule: "0 19 * * *"
  18 │   concurrencyPolicy: Forbid
  19 │   successfulJobsHistoryLimit: 3
  20 │   failedJobsHistoryLimit: 3
  21 │   jobTemplate:
```

Cách đọc lịch cron `"0 19 * * *"`: 5 ô lần lượt là **phút, giờ, ngày trong tháng, tháng, thứ trong tuần**; `*` nghĩa là "mọi". Vậy là "phút 0, giờ 19, mọi ngày". Giờ ở đây là **giờ UTC**, cộng 7 ra giờ Việt Nam: 19 + 7 = 26, tức **2 giờ sáng** hôm sau. `concurrencyPolicy: Forbid`: lần trước chưa xong thì không chạy chồng lần sau.

Script sao lưu backup.sh, từng bước:

hanni-server/infra/k8s/backup/backup.sh, dòng 12–40 (mở trên GitHub):

```
tries=0
until pg_isready --quiet --timeout=2; do
  tries=$((tries + 1))
  [ "$tries" -lt 30 ] || { log "không kết nối được CSDL"; exit 1; }
  sleep 2
done

log "bắt đầu sao lưu CSDL"
pg_dump --format=custom --no-owner --file="$dump.tmp"
pg_restore --list "$dump.tmp" >/dev/null
mv "$dump.tmp" "$dump"

log "bắt đầu sao lưu học liệu"
tar -czf "$media.tmp" -C /srv hoc-lieu
tar -tzf "$media.tmp" >/dev/null
mv "$media.tmp" "$media"

(cd /backup && sha256sum "$(basename "$dump")" "$(basename "$media")" > "hanni-$stamp.sha256")

keep=${BACKUP_KEEP:-14}
for pattern in 'hanni-*.dump' 'hoc-lieu-*.tar.gz' 'hanni-*.sha256'; do
  # shellcheck disable=SC2086
  ls -1t /backup/$pattern 2>/dev/null | tail -n +"$((keep + 1))" | while read -r old; do
    rm -f -- "$old"
  done
done

log "xong: $(du -h "$dump" | cut -f1) CSDL, $(du -h "$media" | cut -f1) học liệu"

```

1. **Chờ CSDL sẵn sàng** (`pg_isready`, tối đa 30 lần × 2 giây). Không tới được thì báo "không kết nối được CSDL" và thoát mã lỗi.
2. **Xuất CSDL** bằng `pg_dump --format=custom` ra tệp tạm `.tmp`, rồi `pg_restore --list` để **tự kiểm tra** tệp đọc được, xong mới đổi tên thành `.dump`. Ghi ra tệp tạm trước nên bản lỗi không bao giờ đè lên bản tốt.
3. **Nén học liệu** bằng `tar -czf` (cũng qua tệp tạm và tự kiểm tra).
4. **Ghi mã băm SHA-256** của 2 tệp vào tệp `.sha256` để sau này kiểm tra tệp có bị hỏng không.
5. **Xoá bản cũ**, chỉ giữ `BACKUP_KEEP` bản mới nhất (14).

Mọi lỗi đều làm script thoát với mã khác 0 (`set -eu`), nên CronJob hiện **thất bại** thay vì âm thầm giữ tệp hỏng.

Đọc nhật ký một lần sao lưu thật (lượt đo chính thức 09/11, chạy tay bằng `create job --from=cronjob/hanni-backup sao-luu-tay`):

data/raw/gha-09-11/ket-qua-kt3/sao-luu/nhat-ky-sao-luu.txt (mở trên GitHub):

```
2026-11-09T12:10:33Z bắt đầu sao lưu CSDL
2026-11-09T12:10:45Z bắt đầu sao lưu học liệu
2026-11-09T12:10:47Z xong: 81.4M CSDL, 7.4M học liệu
```

Mỗi dòng bắt đầu bằng giờ UTC (chữ `Z` ở cuối là UTC; cộng 7 ra giờ Việt Nam). Dòng 1 → dòng 2: xuất CSDL mất 12 giây. Dòng 2 → dòng 3: nén học liệu mất 2 giây. Dòng "xong" ghi dung lượng: CSDL 81,4 MB, học liệu 7,4 MB. **Thiếu dòng "xong" là lần sao lưu đó hỏng**, xem dòng cuối để biết hỏng ở bước nào (ví dụ "không kết nối được CSDL").

Trạng thái job và lịch, đọc bằng `scripts/kubectl -n hanni get cronjob,job`:

data/raw/gha-09-11/ket-qua-kt3/sao-luu/cronjob.txt (mở trên GitHub):

```
NAME                    STATUS     COMPLETIONS   DURATION   AGE
job.batch/sao-luu-tay   Complete   1/1           24s        24s
NAME                          SCHEDULE     TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/hanni-backup    0 19 * * *   <none>     False     0        <none>          23m
```

`COMPLETIONS 1/1` và `STATUS Complete` là thành công (`0/1` , `Failed` là thất bại). `LAST SCHEDULE <none>` vì lab mới dựng 23 phút, chưa tới giờ chạy theo lịch; lần trên là chạy tay. Bản sao lưu được chép ra ngoài cụm rồi kiểm lại mã băm (`sha256sum -c`), `OK` là tệp nguyên vẹn:

data/raw/gha-09-11/ket-qua-kt3/sao-luu/chep-ngoai-cum.txt (mở trên GitHub):

```
hanni-20261109-121031.dump: OK
hoc-lieu-20261109-121031.tar.gz: OK
```

3.4 pg_dump và pg_basebackup (phần mở rộng tự chọn của bạn)

	pg_dump (xuất dữ liệu)	pg_basebackup (chép cả ổ dữ liệu)
Là gì	Xuất nội dung các bảng ra 1 tệp	Chép nguyên thư mục dữ liệu của PostgreSQL
Thời gian sao lưu	12,0 giây	5,0 giây
Dung lượng	82 MB (nén)	556 MB
Thời gian khôi phục	32,7 giây (phải dựng lại chỉ mục)	6,9 giây (chép về là chạy)
Lấy lại 1 bảng	Được	Không
Về đúng thời điểm	Không	Được, kèm nhật ký WAL

Đo trên CSDL có 1 triệu dòng, trung bình 3 lần, số dòng sau khôi phục khớp cả 2 cách. Kết luận: nhóm **dùng cả 2**: bản xuất hằng ngày (nhỏ, chép ra ngoài dễ, lấy lại từng bảng), bản chép cả ổ kèm WAL (khôi phục nhanh, về đúng thời điểm).

3.5 Thử sao lưu khi có lỗi

Script loi-sao-luu.sh cố tình làm hỏng 3 kiểu, kết quả thật:

Lỗi giả lập	Script sao lưu phản ứng	Bản cũ
Đích hết chỗ	Báo "No space left on device", job thất bại	Không bị ảnh hưởng
Đích chỉ đọc	Báo "Read-only file system", job thất bại	Không bị ảnh hưởng
Không tới được CSDL	Báo "không kết nối được CSDL", job thất bại	Không bị ảnh hưởng

> Sao lưu hỏng mà không ai biết còn nguy hiểm hơn không sao lưu. Vì vậy phải thử cả trường hợp lỗi.

Phần 4. Kiểm thử 3 và diễn tập sự cố

4.1 Kiểm thử 3, phần dừng dịch vụ

Script kt3-dung-dich-vu.sh làm 3 lần:

1. Bắt đầu dò API mỗi giây (ghi mã trả về).
2. **Dừng hẳn API** (đưa số bản về 0).
3. Chờ Alertmanager báo cảnh báo, ghi lại sau bao nhiêu giây.
4. **Bật lại** 2 bản, đo bao lâu thì có trả lời.
5. Chạy Kiểm thử 1 để chắc học liệu còn đúng.

Lần	Cảnh báo "từ ngoài không gọi được" kêu sau	Cảnh báo "không còn bản API" kêu sau	Bật lại tới khi có trả lời	Học liệu
1	63 giây	136 giây	5 giây	30/30 tệp đúng
2	84 giây	146 giây	5 giây	30/30 tệp đúng
3	84 giây	146 giây	4 giây	30/30 tệp đúng

Giải thích vì sao cảnh báo kêu sau khoảng 1 phút chứ không ngay lập tức:

hanni-server/infra/observability/rules/hanni-slo.yml, dòng 67–73 (mở trên GitHub):

```
67 |       - alert: HanniEdgeDown
68 |         expr: avg_over_time(probe_success{job="blackbox"}[1m]) < 0.5
69 |         for: 30s
70 |         labels:
71 |           severity: page
72 |         annotations:
73 |           summary: Từ ngoài vào không gọi được API
```

- `expr`: tỉ lệ lần dò thành công **trung bình trong 1 phút** dưới 50%. Dò mỗi vài giây, nên phải hỏng khoảng nửa phút thì trung bình mới xuống dưới 50%.
- `for: 30s`: điều kiện phải đúng liên tục 30 giây nữa mới báo (tránh báo nhầm vì 1 lần chập chờn).
- Cộng lại khoảng 60–90 giây. Đổi được (mục 6.4), nhưng ngắn quá sẽ báo nhầm nhiều.

Thời gian gián đoạn thực tế do người vận hành quyết định (dừng bao lâu thì gián đoạn bấy lâu). Thứ đo được là **phát hiện** (khoảng 1 phút) và **khôi phục** (4–5 giây).

4.2 Kiểm thử 3, phần mất dữ liệu

Script kt3-mat-du-lieu.sh (tự dừng nếu thấy tài khoản không phải dữ liệu giả):

1. CSDL có sẵn 1 triệu dòng nhật ký học, 600 bài thi giả. Chạy sao lưu.
2. **Ghi thêm 1 dòng sau lúc sao lưu** (để chứng minh RPO).
3. **Xoá hẳn CSDL** và **1 tệp học liệu**.
4. Khôi phục từ bản sao lưu.
5. So số dòng 6 bảng chính, kiểm tra dòng ghi thêm ở bước 2, chạy Kiểm thử 1 cho học liệu.

Kết quả: Số dòng 6 bảng chính khớp; dòng ghi sau lúc sao lưu bị mất, đúng như RPO của bản dump; RTO 52 giây từ lúc xoá tới khi kiểm tra xong. Dòng ghi thêm ở bước 2 **bị mất** là **đúng thiết kế**: nó nằm sau lúc sao lưu, chứng minh RPO bằng thời điểm bản sao lưu gần nhất.

4.3 Diễn tập sự cố (chaos)

Script chaos.sh cố tình làm hỏng từng thành phần, đồng thời do-gian-doan.sh gọi API mỗi giây:

Làm hỏng	Gián đoạn dài nhất	Vì sao
Tắt 1 bản API	0 giây	Bản API còn lại vẫn trả lời, Kubernetes tự tạo lại bản đã chết
Tắt Redis	0 giây	Giới hạn request chờ Redis tối đa 150 ms rồi cho qua, API vẫn trả lời
Tắt PostgreSQL	16 giây	Không có CSDL thì API báo chưa sẵn sàng; PostgreSQL tự bật lại thì API tự phục hồi
Tắt 1 node	94 giây	Kubernetes cần thời gian nhận ra node mất rồi mới dời pod sang node còn lại

Cách nói khi bảo vệ: "Mất 1 bản API hay mất Redis thì người dùng không thấy gì. Mất CSDL thì gián đoạn khoảng 16 giây. Mất cả 1 node thì khoảng 1,5 phút, vì lab chỉ có 2 node và Kubernetes cần thời gian nhận ra node đã mất."

4.4 Runbook: khi xảy ra sự cố thì làm gì

Sự cố	Phát hiện bằng	Cách xử lý	Mục tiêu
API ngừng trả lời	Cảnh báo gọi từ ngoài không được	`scripts/kubectl -n hanni get pods`, xem `logs`, bật lại hoặc tăng số bản	Có trả lời lại sau vài giây
Mất dữ liệu CSDL	Lỗi khi đọc ghi, số dòng bất thường	`scripts/restore.sh`: khôi phục bản mới nhất vào CSDL sạch, so số dòng	Mất tối đa 1 ngày (bản dump) hoặc 1 phút (có WAL)
Mất 1 tệp học liệu	Kiểm thử 1 báo tệp sai mã băm	Lấy lại từ gói học liệu đã sao lưu, so mã băm	Học liệu đúng 100%
Đĩa sao lưu đầy	Job sao lưu báo thất bại	Dọn bản cũ hoặc thêm đĩa, chạy lại job	Không để bản lỗi đè bản tốt

Phần 5. Phân quyền và chặn mạng

5.1 Ma trận quyền

Tài nguyên	Khách	Người học	Giáo viên	Quản trị
Danh sách học liệu	Xem	Xem	Xem	Xem
Video bài giảng	Được mời đăng nhập	Xem	Xem	Xem
PDF, bài nghe MP3	Tải	Tải	Tải	Tải
Tiến độ học	Từ chối	Chỉ của mình	Học viên lớp mình	Theo nhu cầu
Trang số liệu nội bộ `/api/metrics`	Không thấy (404)	Không thấy	Không thấy	Chỉ trong cụm
CSDL, Redis	Không tới được	Không tới được	Không tới được	Chỉ trong cụm

5.2 Phân biệt 401, 403, 404, 410 (hay bị hỏi)

- **401**: "bạn là ai?" chưa đăng nhập. Khách xin video cần đăng nhập.
- **403**: "biết bạn là ai nhưng không cho". Người học gọi chức năng quản trị; hoặc liên kết bị sửa chữ ký.
- **404**: "không có thứ này". Dùng để **giấu** `/api/metrics` khỏi người ngoài: nói "không có" an toàn hơn nói "có nhưng cấm".
- **410**: "từng có nhưng hết hạn". Liên kết học liệu quá 10 phút.

5.3 Kiểm thử phân quyền: 9 ca

Script kiem-quyen.sh dùng **tài khoản giả, không dùng quyền quản trị** (quản trị có toàn quyền nên luôn thành công, không chứng minh được gì). Kết quả thật (lượt 09/11):

```
Q1   Khách xin liên kết R1 cần đăng nhập                  mong đợi 401        thực tế 401        ĐẠT
Q2   Dùng liên kết đã hết hạn                             mong đợi 410        thực tế 410        ĐẠT
Q3   Sửa chữ ký trên liên kết                              mong đợi 403        thực tế 403        ĐẠT
Q4   Người học xin liên kết R1 đúng quyền                mong đợi 200        thực tế 200        ĐẠT
Q5   Người học xem tiến độ lớp của người khác     mong đợi 403|404    thực tế 403        ĐẠT
Q6   Máy ngoài cụm nối thẳng Postgres 10.42.1.9:5432        mong đợi dong       thực tế dong
ĐẠT
Q6   Máy ngoài cụm nối Redis qua 127.0.0.1:6379               mong đợi dong       thực tế dong
ĐẠT
Q7   Pod API nối PgBouncer, đường được phép              mong đợi ket-noi    thực tế ket-noi    ĐẠT
Q7   Pod API nối thẳng Postgres, bỏ qua PgBouncer             mong đợi chan       thực tế chan
ĐẠT
Q8   Từ ngoài cụm gọi /api/metrics                          mong đợi 404        thực tế 404
ĐẠT
Q9   Người học gọi route chỉ dành cho quản trị        mong đợi 403        thực tế 403        ĐẠT
```

5.4 Chặn mạng trong cụm (NetworkPolicy)

hanni-server/infra/k8s/policy/network-policy.yaml, dòng 1–13 (mở trên GitHub):

```
  1 | # Mặc định chặn hết chiều vào, chỉ mở đúng đường đi của kiến trúc: ingress → API/media, API → PgBouncer/Redis,
  2 | # PgBouncer → Postgres. Prometheus vào được cổng số đo. API lỡ bị chiếm cũng không chạm thẳng được Postgres.
  3 | apiVersion: networking.k8s.io/v1
  4 | kind: NetworkPolicy
  5 | metadata:
  6 |   name: default-deny-ingress
  7 | spec:
  8 |   podSelector: {}
  9 |   policyTypes: [Ingress]
 10 | ---
 11 | apiVersion: networking.k8s.io/v1
 12 | kind: NetworkPolicy
 13 | metadata:
```

Ý tưởng: **mặc định chặn mọi kết nối vào**, rồi chỉ mở đúng các đường cần: cổng vào tới API và Nginx; API, worker tới Redis và PgBouncer; **chỉ PgBouncer tới PostgreSQL**; Prometheus lấy số liệu. Nhờ vậy, nếu API bị chiếm quyền cũng không nối thẳng được vào PostgreSQL (ca Q7 kiểm tra đúng điều này).

Phần 6. Sửa và cấu hình trực tiếp

Bảng đầy đủ cho cả nhóm ở infra/README.md, mục "Sửa nhanh khi bảo vệ".

6.1 Chạy sao lưu ngay và khôi phục ngay

```
scripts/kubectl -n hanni create job --from=cronjob/hanni-backup sao-luu-tay   # sao lưu ngay, không
chờ 2 giờ sáng
scripts/kubectl -n hanni logs job/sao-luu-tay                                  # xem nhật ký: "xong:
... CSDL, ... học liệu"
scripts/restore.sh                                                             # khôi phục bản mới
nhất vào CSDL sạch, so số dòng
```

`restore.sh` khôi phục vào một CSDL **riêng** tên `hanni_khoi_phuc`, không đụng CSDL đang phục vụ, rồi in số dòng 2 bên để so. Muốn chạy lại job sao lưu tay lần nữa thì đổi tên (`sao-luu-tay-2`).

6.2 Đổi giờ sao lưu

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 16–18 (mở trên GitHub):

```
  16 │   # 19:00 UTC là 02:00 giờ Việt Nam. Giờ chạy trên chính cụm nên chính xác, không trễ như lịch
của GitHub Actions.
  17 │   schedule: "0 19 * * *"
  18 │   concurrencyPolicy: Forbid
```

Ví dụ muốn 23 giờ tối giờ Việt Nam: 23 − 7 = 16 giờ UTC → `"0 16 * * *"`. Chạy `make deploy`, kiểm tra `scripts/kubectl -n hanni get cronjob` (cột SCHEDULE). Mỗi 6 giờ một lần: `"0 */6 * * *"` (RPO giảm từ 1 ngày còn 6 giờ, đổi lại tốn dung lượng hơn).

Thêm ví dụ để tập (ô thứ 5 là thứ trong tuần: 0 = Chủ nhật, 1 = thứ Hai … 6 = thứ Bảy; `*/6` là "cứ 6", `1-5` là "từ 1 tới 5", `1,15` là "1 và 15"):

Muốn (giờ Việt Nam)	Viết (giờ UTC)	Cách nghĩ
02:00 mỗi ngày (hiện tại)	`0 19 * * *`	2 − 7 = −5, cộng 24 = 19 giờ **hôm trước**
23:00 mỗi ngày	`0 16 * * *`	23 − 7 = 16, cùng ngày
12:30 trưa mỗi ngày	`30 5 * * *`	Phút 30, giờ 12 − 7 = 5
Mỗi 30 phút	`*/30 * * * *`	Không phụ thuộc múi giờ
Mỗi 6 giờ	`0 */6 * * *`	Giờ 0, 6, 12, 18 UTC = 7, 13, 19, 1 giờ Việt Nam
02:00 từ thứ Hai tới thứ Sáu	`0 19 * * 0-4`	**Bẫy**: 02:00 thứ Hai ở Việt Nam là 19:00 **Chủ nhật** UTC, nên thứ lùi 1 ngày: 0–4, không phải 1–5
03:00 Chủ nhật hằng tuần	`0 20 * * 6`	03:00 Chủ nhật VN = 20:00 thứ Bảy UTC

Để khỏi phải đổi giờ, CronJob (Kubernetes từ bản 1.27, lab đang dùng 1.31) cho thêm dòng `timeZone: "Asia/Ho_Chi_Minh"` ngay dưới `schedule:`, khi đó viết thẳng giờ Việt Nam (`"0 2 * * *"`). Nhóm giữ giờ UTC vì đó là mặc định và nhật ký cũng ghi giờ UTC, so cho dễ.

6.3 Đổi số bản giữ lại

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 46–47 (mở trên GitHub):

```
46 |                 - name: BACKUP_KEEP
47 |                   value: "14"
```

Đổi `"14"` thành ví dụ `"7"`, `make deploy`. Chạy sao lưu tay vài lần rồi đếm tệp trong thư mục sao lưu: chỉ còn 7 bộ mới nhất.

6.4 Đổi ngưỡng cảnh báo

hanni-server/infra/observability/rules/hanni-slo.yml, dòng 67–69 (mở trên GitHub):

```
67 |       - alert: HanniEdgeDown
68 |         expr: avg_over_time(probe_success{job="blackbox"}[1m]) < 0.5
69 |         for: 30s
```

Ví dụ muốn báo nhanh hơn: đổi `for: 30s` thành `for: 10s`. Chạy `make observability`, rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`. Kiểm tra: dừng API (`scripts/kubectl -n hanni scale deployment hanni-api --replicas=0`), mở http://alertmanager.hanni.localhost:18080, đếm thời gian tới khi cảnh báo hiện; nhớ bật lại (`--replicas=2`).

6.5 Diễn tập 1 sự cố trước mặt thầy

Mở 2 cửa sổ terminal trong `hanni-server/infra`:

```
# cửa sổ 1: dò mỗi giây, Ctrl+C để in thời gian gián đoạn dài nhất
scripts/do-gian-doan.sh k6/ket-qua/dien-tap.csv
# cửa sổ 2: tắt đột ngột 1 bản API
scripts/chaos.sh api
```

Chờ khoảng 30 giây, Ctrl+C ở cửa sổ 1: thấy gián đoạn 0 giây. Mở Grafana thấy Kubernetes tạo lại bản mới.

6.6 Đổi quyền

- Học liệu cần hay không cần đăng nhập: `requiresLogin` trong `k8s/media-manifest.json` (phần của Thành), sau đó `make deploy` + khởi động lại API.
- Ai được nối tới CSDL, Redis: k8s/policy/network-policy.yaml, `make deploy`, chạy lại `scripts/kiem-quyen.sh` để thấy ca kết nối thẳng CSDL vẫn bị chặn.

Phần 7. Đánh giá mở rộng, chạy lại và số liệu

7.1 Đánh giá mở rộng theo số đo

Nguyên tắc: **chỉ thêm khi số đo chạm ngưỡng**, không làm trước. Bảng tính tài nguyên cho từng quy mô:

Chỉ tiêu	100	1.000	10.000	100.000
Tự học cùng lúc	7	63	625	6.250
Xem video cùng giờ	60	300	1.500	5.000
Băng thông video, có dự phòng (Mbps)	178	888	4.441	14.802
Số bản API ước lượng	2	2	5	50

Quy mô	Việc phải làm	Chưa cần	Chuyển mức khi
100	Giới hạn request theo tài khoản; sao lưu đã thử khôi phục	Kubernetes, bản sao, CDN	Máy hết RAM liên tục hoặc p95 đọc vượt 300 ms
1.000	Nhiều bản API tự thêm theo CPU, worker riêng, PgBouncer, video qua Nginx/CDN, giám sát đủ	Chia nhỏ CSDL	Đã đủ 6 bản mà CPU vẫn trên 70%
10.000	CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, hàng đợi việc, chia bảng lớn theo tháng	Chia dữ liệu theo người dùng	CPU CSDL chính trên 70% lúc cao điểm
100.000	Nhiều vùng, tự chuyển CSDL khi hỏng bằng công cụ có sẵn, tách kho phân tích	Tự viết cơ chế đồng thuận	Chi phí và số người vận hành

Lý do dựa trên điểm nghẽn đã đo: ở 1.000, điểm nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong chuyển sang CPU của API (thêm bản là xong); lên 10.000 thì CSDL thành điểm nghẽn, phải tách máy riêng và có bản sao; video thì luôn phải qua CDN.

7.2 Chạy lại phần của bạn, từng bước

Mọi lệnh chạy trong `~/hanni-server/infra` khi lab đang chạy (mục 1.6). Cột "Thấy gì là đúng" lấy từ đầu ra thật của lượt đo chính thức 09/11, số của bạn có thể khác chút ít; thấy khác hẳn thì xem cột cuối, rồi mục 1.8.

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
1. Dựng lab	Mục 1.6	`make status`: mọi pod `Running` hoặc `Completed`	Mục 1.8
2. Sao lưu ngay	`scripts/kubectl -n hanni create job --from=cronjob/hanni-backup sao-luu-tay`, rồi `scripts/kubectl -n hanni wait --for=condition=complete job/sao-luu-tay --timeout=180s`	`job.batch/sao-luu-tay created`, rồi `job.batch/sao-luu-tay condition met`	Báo tên đã có: đổi thành `sao-luu-tay-2`

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
3. Đọc nhật ký sao lưu	`scripts/kubectl -n hanni logs job/sao-luu-tay`	3 dòng như mục 3.3, dòng cuối `xong: 81.4M CSDL, 7.4M học liệu` (dung lượng có thể khác)	Thiếu dòng `xong` : đọc dòng cuối để biết hỏng ở bước nào
4. Khôi phục thử	`scripts/restore.sh`	`Bản sao lưu: /backup/hanni-....dump` , các dòng `OK` , bảng 6 bảng với số "đang chạy" bằng số "khôi phục", `Thời gian khôi phục: … giây`	Có dòng `Lệch số dòng` : có ghi mới sau lúc sao lưu, phải giải thích được
5. Kiểm thử 3, dừng dịch vụ	`scripts/kt3-dung-dich-vu.sh k6/ket-qua/kt3 3` (khoảng 10 phút)	Mỗi lần 1 dòng `lần 1: biên 63 giây, API 136 giây, bật lại 5 giây, gián đoạn 139 giây, học liệu dung`	`học liệu sai` : xem `k6/ket-qua/kt3/lan-1-hoc-lieu.txt`
6. Kiểm thử 3, mất dữ liệu	`scripts/kt3-mat-du-lieu.sh`	`Số dòng: KHỚP` , `Dòng ghi sau lúc sao lưu còn tồn tại: 0` , `Tổng: 30 tệp đúng, 0 tệp sai` , `RTO, từ lúc xoá tới khi kiểm tra xong: 52 giây`	
7. Phân quyền	`scripts/kiem-quyen.sh k6/ket-qua/quyen.txt`	Mỗi dòng Q1–Q9 kết thúc bằng `ĐẠT` , dòng cuối `Tất cả ca đạt`	
8. Diễn tập mất 1 bản API	Tab 1: `scripts/do-gian-doan.sh k6/ket-qua/api.csv` ; tab 2: `scripts/chaos.sh api` ; chờ 30 giây, Ctrl+C ở tab 1	Tab 1 in `… lượt, 0 lượt lỗi, gián đoạn dài nhất 0 giây`	
9. Thử lỗi sao lưu	`scripts/loi-sao-luu.sh k6/ket-qua/loi-sao-luu.txt`	3 dòng `job thất-bại … ĐẠT` , dòng cuối `Bản sao lưu cũ không bị ảnh hưởng: ĐẠT`	

7.3 Số liệu để nói khi bảo vệ

Câu cần nói	Số
Dừng API: cảnh báo kêu sau	63, 84, 84 giây
Bật lại: có trả lời sau	5, 5, 4 giây; học liệu 30/30 đúng cả 3 lần
Mất dữ liệu	Số dòng khớp, RTO 52 giây, dòng ghi sau sao lưu mất (đúng RPO)
Tắt từng thành phần	mất 1 bản API 0 s, mất Redis 0 s, mất Postgres 16 s, mất 1 node 94 s
Sao lưu pg_dump / pg_basebackup	82 MB, khôi phục 33 giây / 556 MB, 7 giây
Thử lỗi sao lưu	3/3 trường hợp báo lỗi đúng, bản cũ không bị ảnh hưởng
Phân quyền	11/11 dòng đạt

7.4 README của hồ sơ: viết gì, vì sao, kiểm thế nào

Phiếu đề yêu cầu README ghi **phiên bản, cách dựng lại, cách chạy kiểm thử**. Bạn phụ trách ho-so-nop/README.md và infra/README.md. Mỗi mục trong README hồ sơ trả lời 1 câu người chấm sẽ hỏi:

Mục	Trả lời câu	Khi nào phải sửa
Đầu tệp, bảng thành viên	Lớp, nhóm, ai làm phần nào, mã sinh viên	Khi biết số nhóm
Nội dung hồ sơ	Mỗi thư mục, tệp nộp chứa gì	Thêm hoặc bớt tệp nộp
Mã nguồn đầy đủ	Repo, nhánh, **commit** nào đúng với hồ sơ	Mỗi lần đẩy commit mới lên nhánh
Phiên bản	Docker, k3s, Node.js, PostgreSQL, k6... bản nào	Đổi image hoặc công cụ
Yêu cầu máy	Máy thế nào thì dựng được	Lab nặng hơn
Dựng lại hệ thống	6 lệnh `make`, thời gian dựng, cách chạy web	Đổi Makefile
Chạy kiểm thử	Mỗi ca: lệnh, kết quả nằm ở đâu, tiêu chí đạt	Thêm ca kiểm thử
Sửa cấu hình khi bảo vệ	Trỏ tới bảng "Sửa nhanh" trong infra/README.md	Thêm thông số hay bị hỏi
Nguồn tham khảo, dùng AI	Nguồn dữ liệu, giấy phép, nhóm dùng trợ lý AI thế nào	Thêm nguồn

Kiểm README đúng hay chưa không phải bằng đọc lại, mà bằng **làm theo trên máy sạch**: mỗi lượt đo chính thức, máy ảo GitHub mới tinh dựng lab chỉ bằng đúng các lệnh trong README (nhật ký ở evidence/dung-lai/ và `data/raw/gha-09-11/ket-qua-*/dung-lab.txt`). Lệnh nào trong README sai thì lượt đó hỏng ngay. Muốn tự thử: clone repo về thư mục mới, làm đúng từng dòng README, ghi lại chỗ vướng. Quy tắc viết: mỗi lệnh chép-dán chạy được; ghi kết quả mong đợi ("mọi pod Running"); không ghi mật khẩu thật (mật khẩu lab sinh ngẫu nhiên lúc `make up`).

Phần 8. Câu hỏi vấn đáp

Trả lời ngắn, có số, rồi mở tệp ghi ở dòng **Mở minh chứng** dưới mỗi câu. Thầy hỏi khác đi một chút thì vẫn dựa vào ý chính của câu trả lời.

Câu hỏi về phần của bạn

RPO khác RTO thế nào?

RPO là lượng dữ liệu tối đa có thể mất (tính từ bản sao lưu gần nhất); RTO là thời gian từ lúc hỏng tới lúc dùng lại được. Sao lưu 1 lần/ngày thì RPO tối đa 1 ngày dù khôi phục nhanh đến đâu.

Mở minh chứng: data/rpo-rto.xlsx

Vì sao bản sao lưu phải để nơi khác?

Hỏng máy hoặc ổ đĩa sẽ mất cả bản chính lẫn bản sao nếu để chung. Nhóm chép ra ngoài cụm và so mã băm; giới hạn là trong lab vẫn cùng 1 máy.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/sao-luu/

Dừng API thì có mất dữ liệu không?

Dữ liệu đã ghi nằm trong PostgreSQL nên không mất; request đang xử lý có thể lỗi. 3 lần dừng: cảnh báo kêu sau 63, 84, 84 giây, bật lại có trả lời sau 5, 5, 4 giây, học liệu đúng 30/30.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/kt3-a/

Vì sao kiểm tra quyền không dùng tài khoản quản trị?

Quản trị có toàn quyền nên luôn thành công, không chứng minh được gì. Phải dùng đúng vai trò: khách, người học.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/quyen.txt

Sao lưu xuất dữ liệu khác chép cả ổ thế nào?

pg_dump: 82 MB, khôi phục 33 giây, lấy lại được từng bảng. pg_basebackup: 556 MB, khôi phục 7 giây, kết hợp nhật ký thay đổi để về đúng thời điểm. Nhóm dùng cả 2.

Mở minh chứng: Báo cáo nhóm mục 5.7

Thao tác có thể bị yêu cầu

Chạy sao lưu tay rồi restore.sh; đổi giờ sao lưu trong k8s/backup/cronjob.yaml (giờ UTC); đổi số bản giữ BACKUP_KEEP; tắt 1 bản API bằng scripts/chaos.sh api và đọc thời gian gián đoạn.

Mở minh chứng: cau-hinh-ma-nguon/hanni-server/infra/README.md, mục "Sửa nhanh khi bảo vệ"

Câu hỏi chung ai cũng phải trả lời được

Vì sao thử 20 kết nối không chứng minh phục vụ được 1.000 sinh viên?

Thời gian trả lời không tăng đều: từ 10 tới 40 lượt/giây p95 chỉ từ 23 lên 52 ms, thêm 10 lượt nữa đã 261 ms. Mức nhỏ chưa chạm giới hạn nào nên không nói gì về điểm gãy. Kết quả chỉ đúng cho đúng máy, dữ liệu, cấu hình đã đo.

Mở minh chứng: Báo cáo nhóm mục 5.3 (đo thăm dò), 6.2

Tài khoản khác người dùng cùng lúc thế nào?

Tài khoản là số người có thể dùng; người dùng cùng lúc mới tạo tải. 1.000 tài khoản sinh ra 63 người khi tự học, 300 khi cả lớp xem video, 600 khi thi thử đồng loạt.

Mở minh chứng: Báo cáo nhóm mục 2.1–2.2; so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx

Vì sao video không đi qua API?

Cả lớp xem video cần khoảng 888 Mbps trong khi API chỉ khoảng 56 request/giây. API chỉ kiểm quyền và ký liên kết, tệp do Nginx (thực tế là CDN) phát.

Mở minh chứng: Báo cáo nhóm mục 3.2; cau-hinh-ma-nguon/hanni-server/infra/k8s/app/media.yaml

Thêm máy chủ có hết quá tải không?

Không, nếu điểm nghẽn ở CSDL hoặc băng thông. Phải đo để biết điểm nghẽn trước: ở lab, nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong mới tới CPU của API.

Mở minh chứng: Báo cáo nhóm mục 6.1; bảng Grafana "Mở rộng"

Bản sao lưu thế nào mới có giá trị?

Khi đã khôi phục thử, so số dòng và mã băm, đo được thời gian. Nhóm đã xoá hẳn CSDL rồi khôi phục: số dòng khớp, RTO 52 giây.

Mở minh chứng: Báo cáo nhóm mục 5.4; data/rpo-rto.xlsx

Lên 10.000 hay 100.000 sinh viên thì làm gì?

10.000: CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, khoảng 5 bản API. 100.000: nhiều vùng, tự chuyển CSDL khi hỏng, tách kho phân tích. Chuyển mức khi số đo chạm ngưỡng, không làm trước.

Mở minh chứng: Báo cáo nhóm mục 6.4

Phần 9. Việc cần làm trước khi nộp và bảo vệ

Trước khi nộp

Báo cáo cá nhân của bạn `ho-so-nop/bao-cao/ca-nhan/RIPT1304-20261-01_NhomXX_06_B24DCCC065_BaoCaoCaNhan.docx` **đã điền đủ** theo mẫu: mã sinh viên, lớp, ngày, họ tên ở mục 10. Chỉ còn số nhóm (chỗ `XX` trong tên tệp và dòng "Nhóm"), Đăng điền khi thầy công bố rồi nộp cả hồ sơ lên Drive của thầy.

Việc của bạn: đọc báo cáo nhóm `ho-so-nop/bao-cao/Bao-cao-nhom.pdf` và báo cáo cá nhân của bạn để nắm mình đã kê khai những gì.

Trình bày 3 phút

Lúc bảo vệ, mỗi người có vài phút trình bày phần mình. Khung dưới đây đi đúng thứ tự thầy chấm: việc gì, bằng chứng ở đâu, số đo bao nhiêu, giới hạn là gì. Tập nói to 2–3 lần, bấm giờ.

Thời gian	Nói gì	Mở gì
0:00–0:40	Sao lưu hằng ngày 02:00: pg_dump CSDL + nén học liệu, tự kiểm tra tệp, ghi mã băm, giữ 14 bản, chép ra ngoài cụm; lỗi thì job báo thất bại (3/3 tình huống lỗi)	k8s/backup/cronjob.yaml, nhật ký sao lưu
0:40–1:30	Kiểm thử 3 dừng dịch vụ: dừng hẳn API 3 lần, cảnh báo từ ngoài sau 63, 84, 84 giây, bật lại có trả lời sau 5, 5, 4 giây, học liệu đúng	ket-qua-kt3/kt3-a/
1:30–2:10	Kiểm thử 3 mất dữ liệu: xoá CSDL và 1 tệp học liệu, khôi phục, số dòng khớp, RTO 52 giây; RPO tối đa 1 ngày với bản dump (dòng ghi sau lúc sao lưu mất đúng như dự kiến)	data/rpo-rto.xlsx
2:10–2:40	Diễn tập: mất 1 bản API 0 s, mất Redis 0 s, mất Postgres 16 s, mất 1 node 94 s; phân quyền 11/11 dòng đạt	ket-qua-kt3/chaos/, quyen.txt
2:40–3:00	Mở rộng theo điểm nghẽn: 1.000 → thêm bản API + CDN; 10.000 → CSDL riêng có bản sao; giới hạn: lab 1 máy	Mục 7.1

Trước buổi bảo vệ

1. Tập **mở đúng minh chứng trong 10 giây**: biết tệp kết quả của mình nằm ở đâu trong `ho-so-nop/` (bảng "Sản phẩm của bạn" ở phần 2).
2. Tập **1–2 thao tác** ở phần "Sửa và cấu hình trực tiếp": nói được tệp nào, dòng nào, lệnh áp dụng, cách kiểm tra.
3. Tập trả lời các câu ở phần Câu hỏi vấn đáp bằng lời của mình, có số.
4. Xem lại phần 1 (kiến thức nền): thầy có thể hỏi kiến trúc chung, không chỉ phần của bạn.

> Câu chốt khi bị hỏi khó: "Theo số đo của nhóm trên lab, trong điều kiện … thì … ; giới hạn là … ; muốn chắc hơn thì đo thêm …". Trả lời theo số đo và nêu giới hạn luôn được đánh giá cao hơn khẳng định chung chung.